

AI 제조 기술 관련 교육

『3과목』 RAG & 고급 프롬프트

2025.12.08-2025.12.19(10일 70시간)

Prepared by Daekyeong Kim

Ph.D.

Copyright 2022. Daekyeong all rights reserved

『3과목』

RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
- **Part 4. AI 에이전트 개발**
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약





학습목표

- 이 워크샵에서는 AI 에이전트 개발과 오픈AI의 에이전트 SDK에 대해 알 수 있다

눈높이 체크

- Agents 를 알고 계시나요?



1. Agents?

Agents의 기본 개념

- 랭체인(LangChain)의 Agents는 다양한 작업을 수행하기 위해 능동적으로 의사결정을 내리고 행동할 수 있는 시스템을 의미합니다. Agents는 주어진 목표를 달성하기 위해 여러 도구와 체인을 결합하여 다양한 작업을 처리할 수 있는 강력한 기능을 제공합니다. 이를 통해 더욱 복잡한 문제를 해결하고, 다양한 데이터 소스나 API와 상호작용할 수 있는 유연한 워크플로우를 만들 수 있습니다.
- LLM(대형 언어 모델): 자연어 처리와 의사결정의 기본 엔진으로 사용됩니다.
- Chain(체인): 여러 작업을 조합하여 복잡한 워크플로우를 구성할 수 있는 체인입니다.



1. Agents?

Agents의 주요 구성 요소

- Tools(도구): 계산, 검색과 같이 "언어 모델만으로는 할 수 없는 일"을 할 수 있게 하는 모듈로, 검색, 계산, API 호출, 데이터 변환 등의 기능을 수행합니다.
 - 랭체인이 제공하는 Tool로는 LLMMath(계산), Requests(URL 호출), File System(로컬 파일 접근), SerpApi(검색) 등이 있다.
 - 랭체인이 제공하는 것 외에도, 직접 만든 Tool을 사용할 수 있다.
 - Agent의 역할: Agents는 사용자가 지정한 목표를 달성하기 위해 스스로 결정을 내리고, 적절한 도구나 체인을 선택하여 작업을 수행하는 엔티티입니다. Tool을 선택하고 다음 단계의 처리를 수행하는 주체로, 단순히 Tool을 조작하는 것이 아닌, 스스로 어떤 Tool을 어떻게 사용하면 좋을지 고민하고 실행, 결과 검증까지 진행한다.
 - Agent에서 가장 많이 사용하는 'ReAct' 기법이 있다.



1. Agents?

Agents의 동작 원리

- 입력: 사용자가 에이전트에게 작업 요청을 전달합니다.
- 처리: 에이전트는 LLM을 사용하여 입력을 분석하고, 어떤 도구나 체인을 사용할지 결정합니다.
- 실행: 필요한 도구나 체인을 실행하여 결과를 생성합니다.
- 출력: 최종 결과를 사용자에게 반환합니다.



1. Agents?

Agents의 유형

- 랭체인에서는 다양한 유형의 Agents를 제공하여 사용자가 요구하는 다양한 작업을 수행할 수 있도록 합니다.
- Zero-Shot React Description
 - 설명: 주어진 작업에 대해 직접적이고 간단한 응답을 생성하는 에이전트입니다. 외부 도구를 사용하지 않고 언어 모델의 응답만으로 결과를 생성합니다.
 - 사용 사례: 간단한 질의응답, 빠른 텍스트 생성.
- ReAct
 - 설명: ReAct는 주어진 목표를 달성하기 위해 복합적인 작업을 수행하는 에이전트입니다. 이 에이전트는 상황에 따라 필요할 때마다 도구를 호출하거나 체인을 실행할 수 있습니다.
 - 사용 사례: 질문에 대한 답변을 위해 외부 정보를 검색하거나, 계산을 수행해야 하는 복잡한 시나리오.



1. Agents?

Agents의 유형

● Self-Ask with Search

- 설명: 질문을 처리하는 과정에서 스스로 추가적인 질문을 생성하여 검색을 수행하는 에이전트입니다.
- 사용 사례: 정확한 정보 검색이 필요한 상황에서 유용하며, 추가적인 세부사항을 알아야 할 때 활용됩니다.

● MRKL (Modular Reasoning, Knowledge and Language)

- 설명: MRKL 에이전트는 복잡한 의사결정과정을 수행하기 위해 다양한 모듈을 조합하여 동작하는 구조를 가지고 있습니다.
- 사용 사례: 특정 도메인 지식이 필요하고, 여러 개의 도구와 데이터 베이스를 활용해야 하는 상황에서 사용.

● Conversational Agent

- 설명: 대화형 에이전트로, 대화의 맥락을 이해하고, 필요한 경우 이전의 발화를 참고하여 답변을 생성합니다.
- 사용 사례: 챗봇, 고객 지원 시스템 등에서 사용자와의 상호작용을 위해 사용.



1. Agents?

Agents의 활용 예시

- 질의응답 시스템: 사용자로부터 질문을 받고, 관련 정보를 검색하고, 필요에 따라 데이터를 분석하거나 요약한 뒤 응답을 생성하는 시스템.
- 자동화 도구: 다양한 API를 호출하거나 데이터베이스와 상호작용하여 데이터를 가져오고, 이를 가공하여 사용자에게 유용한 정보를 제공하는 도구.
- 복잡한 문제 해결: 여러 단계로 이루어진 복잡한 문제를 해결하기 위해, 각 단계마다 필요한 작업을 정의하고, 이를 자동으로 수행하는 에이전트.



1. 실습

| 실습 17: 인사하는 에이전트

`hello_agent_async.py`

```
pip install openai-agents==0.2.5
```



2. Tool을 추가해 Agents 구현

Tool

- Agent가 할 수 있는 것은 전달하는 Tool에 따라 달라진다
- File System(로컬 파일 접근): WriteFileTool은 파일 쓰기를 할 수 있는 Tool이다.
- SerpApi(검색): 구글 등 검색 엔진의 검색 결과를 API로 가져올 수 있는 서비스
- <https://serpapi.com/manage-api-key> 에서 API키를 발급 받을 수 있다.
- AgentType를
STRUCTURED_CHAT_ZERO_SHOT_REACT_DESCRIPTION으로 변경하였는데, CHAT_ZERO_SHOT_REACT_DESCRIPTION와의 차이점은, 여러 개의 입력이 필요한 상황에서 작동한다는 점이다 (CHAT_ZERO_SHOT_REACT_DESCRIPTION은 단일 입력).



2. Tool을 추가해 Agents 구현

SerpApi

- SerpApi는 검색 엔진 결과 페이지(SERP: Search Engine Results Page)의 데이터를 수집하고 분석할 수 있는 API 서비스입니다. SerpApi는 Google, Bing, Yahoo, Yandex, Baidu 등 여러 검색 엔진에서 검색 결과를 수집하여 JSON 형식으로 반환해주는 API를 제공합니다. 이를 통해 개발자는 웹 스크래핑 없이 손쉽게 검색 엔진의 결과를 사용할 수 있습니다.

2. Tool을 추가해 Agents 구현

SerpApi의 주요 기능 및 활용

- 다양한 검색 엔진 지원: Google, Bing, Yahoo 등 주요 검색 엔진 외에도 지역별 검색 엔진, 이미지 검색, 뉴스 검색 등을 지원합니다.
- 세밀한 결과 추출: 검색 결과 페이지의 제목, 설명, URL, 이미지, 광고 등 다양한 정보를 정확하게 추출할 수 있습니다.
- 개발 편의성: 간단한 API 호출을 통해 원하는 정보를 얻을 수 있으며, 다양한 프로그래밍 언어에서 사용할 수 있는 SDK를 제공합니다.
- 활용 분야:
 - SEO 분석: 키워드 순위 추적, 경쟁 분석, 검색 엔진 알고리즘 연구 등
 - 데이터 수집: 특정 주제에 대한 방대한 양의 데이터를 수집하여 분석
 - 챗봇 개발: 사용자의 질문에 대한 답변을 검색 엔진에서 찾아 제공
 - 시장 조사: 제품, 브랜드, 트렌드 등에 대한 정보를 수집하여 시장 분석
 - 학술 연구: 특정 주제에 대한 연구 자료를 수집하고 분석

2. Tool을 추가해 Agents 구현

SerpApi Google Search API 발급 받기

- <https://serpapi.com/>에서 Register에서 회원가입 후 사용

The screenshot displays the SerpApi website interface. At the top, there's a navigation bar with links for Documentation, Integrations, Features, Pricing, Use cases, Team, FAQ, and Contact us. A 'Sign In' button and a 'Register' button are also present. The main heading is 'Google Search API', followed by the tagline 'Scrape Google and other search engines from our fast, easy, and complete API.' Below this, there's a search interface with a 'Search Query' field containing 'Coffee' and a 'Location' dropdown menu set to 'Austin, Texas, United States'. A green 'TEST SEARCH' button is to the right. The search results are shown in a preview window, displaying a Google search page for 'Coffee' with a map of coffee shops in Austin, Texas, and a list of results including Starbucks and Houndsborth Coffee. To the right of the preview, a JSON response is shown, detailing the search metadata and parameters used.

```
{  "search_metadata": {    "id": "6165916694c6c7025deef5ab",    "status": "Success",    "json_endpoint": "https://serpapi.com/searches/87fa874d05a7fcc5/6165916694c6c7025deef5ab.json",    "created_at": "2021-10-12 13:45:10 UTC",    "processed_at": "2021-10-12 13:45:11 UTC",    "google_url": "https://www.google.com/search?q=Coffee&oeq=Coffee&uile=wCAIQCiaQXVzdGluFR1eGFz",    "raw_html_file": "https://serpapi.com/searches/87fa874d05a7fcc5/6165916694c6c7025deef5ab.html",    "total_time_taken": 1.85  },  "search_parameters": {    "engine": "google",    "q": "Coffee",    "location_requested": "Austin, Texas, United States",    "location_used": "Austin, Texas, United States",    "google_domain": "google.com",    "hl": "en",    "gl": "us",    "device": "desktop"  },}
```



2. Tool을 추가해 Agents 구현

특정 숫자 이상의 임의의 숫자를 생성하는 Tool

- Tool을 직접 만들어서 할 수 있는 일의 폭을 더욱 넓힌다
- 랭체인에서 제공하는 기본적인 Tool뿐만 아니라, 사용자가 직접 Tool을 제작할 수 있다.
- Tool은 기본적으로 '이름', '기능 설명', '실행 함수' 3가지 요소로 구성되어 있으며, 사용자의 Custom Tool도 같은 형태로 작성되어야 한다.

2. Tool을 추가해 Agents 구현

Retrievers를 사용해 문장 검색 Tool 만들기

- Retrievers를 Tool로 변환해 Agent에서 사용하도록 할 수 있다(위키피디아, 벡터DB에서 검색).
- WikipediaRetriever를 Tool로 지정(직접 만든 것은 아니고, create_retriever_tool 함수에서 미리 정의해둔 코드가 있다)

```
retriever = WikipediaRetriever(           ←WikipediaRetriever를 초기화
    lang="ko",           ←언어를 한국어로 설정
    doc_content_chars_max=500,           ←글의 최대 글자 수를 500자로 설정
    top_k_results=1      ←검색 결과 중 상위 1건을 가져옴
)
```




2. Tool을 추가해 Agents 구현

Retrievers를 사용해 문장 검색 Tool 만들기

- 명령문 입력

```
result = agent.run("스카치 위스키에 대해 Wikipedia에서 찾아보고 그 개요를 한국어로  
result.txt 파일에 저장하세요.")
```

- 여러개의 Document를 가져와서

➤ Entering new AgentExecutor chain...

Thought: I will use the WikipediaRetriever tool to search for information about Scotch whisky and then write a summary in Korean to a file named "result.txt".

Action:

```
```\n{\n  "action": "WikipediaRetriever",\n  "action_input": "Scotch whisky"\n}\n```\n
```

Observation: [Document(page\_content='위스키(영어: whisky 또는 whiskey, 문화어: 위스끼)는 맥아 및 기타 곡류를 당화 발효시킨 발효주를 증류하여 만든 술이다. 주로 보리, 옥수수, 호밀, 밀 등의 곡물이

## 2. Tool을 추가해 Agents 구현

### Retrievers를 사용해 문장 검색 Tool 만들기

- Retrieval task를 진행

```
Thought:Action:
...
{
 "action": "write_file",
 "action_input": {
 "file_path": "result.txt",
 "text": "위스키(영어: whisky 또는 whiskey, 문화어: 위스끼)는 맥아 및 기타 곡류를 당화 발효시킨 발효주를 증류하여 만든 술이다. 주로 보리, 옥수수, 호밀, 밀 등의 곡물이 원료가 된다. 증류 후에는 L",
 "append": false
 }
}
...
Observation: File written successfully to result.txt.
...

> Finished chain.
실행 결과: 작업이 성공적으로 완료되었습니다.
```

- 결과를 result.txt에 저장

```
6장.ipynb U ● result.txt X
langchain > 06_agent > result.txt
1 위스키(영어: whisky 또는 whiskey, 문화어: 위스끼)는 맥아 및 기타 곡류를 당화 발효시킨 발효주를 증류하여 만든 술
```



## 2. Tool을 추가해 Agents 구현

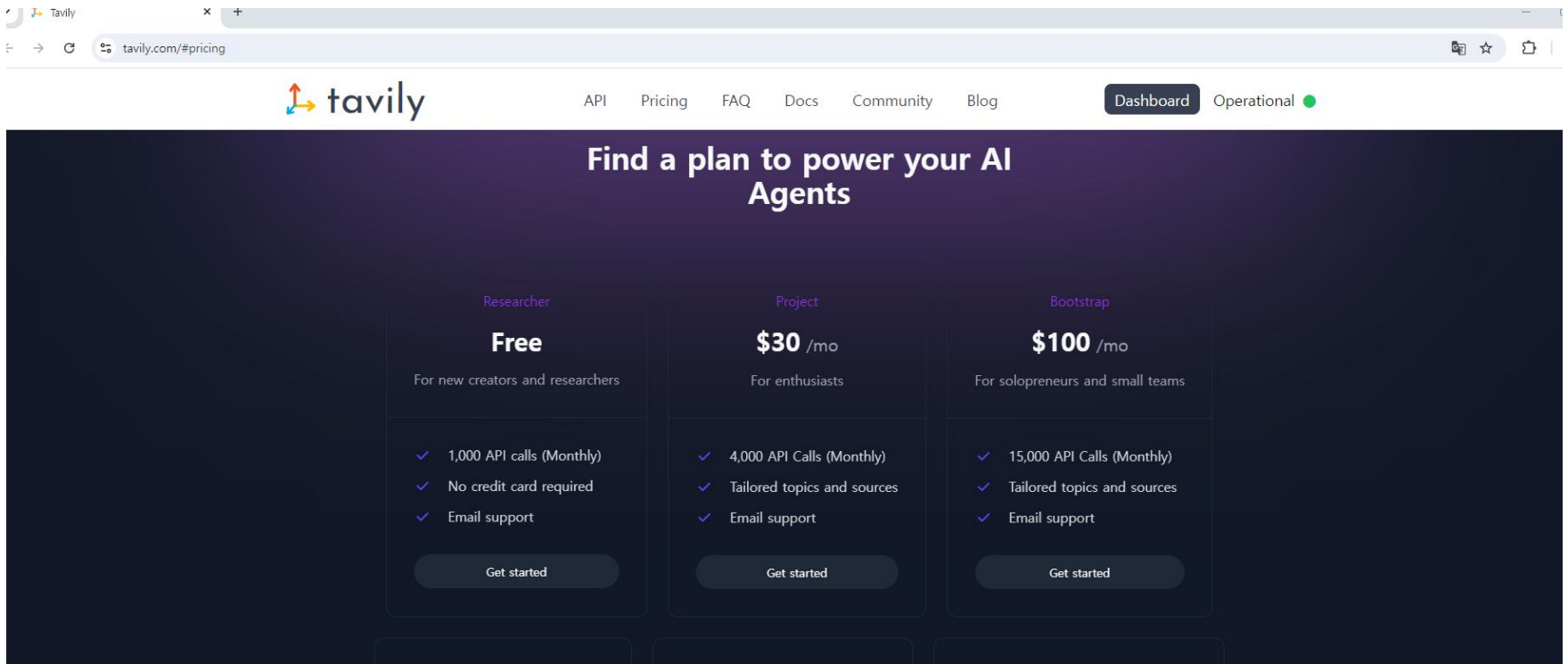
### PDF 기반 문서 검색 도구: Retriever

- 우리의 데이터에 대해 조회를 수행할 retriever도 생성합니다.
- 이 코드는 웹 기반 문서 로더, 문서 분할기, 벡터 저장소, 그리고 OpenAI 임베딩을 사용하여 문서 검색 시스템을 구축합니다.

## 2. Tool을 추가해 Agents 구현

### 도구를 활용한 토론 에이전트(Agent Debates with Tools)

- <https://tavily.com/#pricing>



The screenshot shows the Tavily website's pricing page. The browser address bar displays "tavily.com/#pricing". The website header includes the Tavily logo, navigation links (API, Pricing, FAQ, Docs, Community, Blog), a "Dashboard" button, and a status indicator "Operational" with a green dot. The main heading reads "Find a plan to power your AI Agents". Below this, three pricing plans are presented in a grid:

Researcher	Project	Bootstrap
<b>Free</b> For new creators and researchers	<b>\$30 /mo</b> For enthusiasts	<b>\$100 /mo</b> For solopreneurs and small teams
✓ 1,000 API calls (Monthly)    ✓ No credit card required    ✓ Email support	✓ 4,000 API Calls (Monthly)    ✓ Tailored topics and sources    ✓ Email support	✓ 15,000 API Calls (Monthly)    ✓ Tailored topics and sources    ✓ Email support
<a href="#">Get started</a>	<a href="#">Get started</a>	<a href="#">Get started</a>



## 2. 실습

### | 실습 18: 뉴스 에이전트 만들기

- `news_search_agent.py`



## 2. 실습

### | 실습 19: 가드레일 사용하기

- `input_output_guardrail_test.py`



## 2. 실습

### | 실습 20: 핸드오프 활용

- `simple_multi_agent_by_handoff.py`

### 3. AI 에이전트, 새로운 패러다임의 시작

#### | 왜 지금 AI 에이전트인가?

- 단순한 챗봇을 넘어: 사용자의 요청을 이해하고, 계획을 세우고, 스스로 행동하는 AI
  - AI 에이전트의 산업별 활용 사례
  - 고객 지원 자동화: 24시간 고객 문의 응대 및 문제 해결
  - 데이터 분석 자동화: 복잡한 데이터 조회 및 리포트 생성
  - 개인 비서: 이메일 정리, 일정 관리, 정보 검색 등



### 3. AI 에이전트, 새로운 패러다임의 시작

#### AI 에이전트의 핵심 원리: ReAct 프레임워크

- ReAct: Reason(추론) + \*\*Act(행동)\*\*의 결합
- LLM이 단순히 답변을 생성하는 것을 넘어, '생각'하고 '행동'하게 만드는 방법
- 작동 방식 (순환 구조)
  - Thought (생각): 사용자의 목표를 달성하기 위해 어떤 행동이 필요한지 추론한다.
  - Action (행동): 가장 적절한 도구(Tool)를 선택하고 실행한다.
  - Observation (관찰): 도구 실행 결과를 관찰(입력)받는다.
  - Thought (생각): 관찰된 결과를 바탕으로 다음 행동을 다시 추론하거나, 최종 답변을 생성한다.

### 3. AI 에이전트, 새로운 패러다임의 시작

#### | Google ADK 소개

- ADK란?: Gemini 모델을 기반으로 강력하고 확장 가능한 AI 에이전트를 쉽게 구축할 수 있도록 돕는 개발 프레임워크
- ADK의 장점:
  - Tool 사용의 단순화: 외부 API나 함수를 '도구'로 쉽게 등록하고 LLM이 사용하도록 할 수 있음.
  - 추론 능력 강화: ReAct 패턴을 내장하여 복잡한 문제 해결 능력을 극대화.
  - Google 생태계 연동: Google 검색, Google Workspace 등 다양한 구글 서비스와 손쉽게 연동 가능.

### 3. AI 에이전트, 새로운 패러다임의 시작

#### | LLM: 에이전트의 '두뇌'

- 에이전트의 모든 추론과 계획을 담당하는 핵심 엔진 (예: Gemini)
- 사용자의 의도를 파악하고, 어떤 도구를 사용해야 할지 결정하는 역할

### 3. AI 에이전트, 새로운 패러다임의 시작

#### Tools: 에이전트의 '손과 발'

- 에이전트가 세상과 상호작용할 수 있게 해주는 모든 수단.
- 예시:
  - 특정 웹사이트의 정보를 가져오는 함수
  - 데이터베이스에서 데이터를 조회하는 API
  - 사용자에게 이메일을 보내는 기능

# 3. AI 에이전트, 새로운 패러다임의 시작

## Tools: 에이전트의 '손과 발'

- Tool 정의 방법: Python 함수에 @tool 데코레이터만 추가하면 간단하게 정의 가능.

# 예시: 날씨 정보를 가져오는 Tool  
from adk.tools import tool

```
@tool
def get_weather(city: str) -> str:
 """주어진 도시의 현재 날씨 정보를 알려줍니다."""
 # ... (실제 날씨 API 호출 코드) ...
 return f"{city}의 현재 날씨는 맑고, 기온은 25도입니다."
```

### 3. AI 에이전트, 새로운 패러다임의 시작

#### Agent: '두뇌'와 '손과 발'을 연결하는 실행자

- 사용자의 입력을 받아 LLM에게 전달하고, LLM의 계획에 따라 Tool을 실행시키는 컨트롤 타워.
- ADK에서는 Agent 클래스를 사용하여 간단하게 생성 가능.

# 예시: Agent 생성 및 실행

```
from adk.agent import Agent
from adk.llm import Gemini
```

# 1. 에이전트의 두뇌(LLM) 정의

```
llm = Gemini()
```

# 2. 에이전트가 사용할 도구들 정의

```
tools = [get_weather]
```

# 3. 에이전트 생성

```
agent = Agent(llm=llm, tools=tools)
```

# 4. 에이전트 실행

```
result = agent.run("오늘 서울 날씨 어때?")
print(result)
```



## 4. LangGraph?

### LangGraph?

- Agent 와 도구(Tool)를 이용하면 LLM 의 추론능력으로 자동화된 작업 처리능력을 얻을 수 있습니다. 사용자의 문제를 해결하기 위하여 Agent는 자신에게 주어진 도구들을 이용해서 일련을 작업들을 구성해냅니다.
- 이런 강력한 Agent 도 만능은 아닙니다. 하나의 Agent 가 많은 도구를 사용하여 사이즈가 큰 작업을 처리해야하는 경우 Agent 의 성능이 저하됩니다. 따라서 Agent 를 기능/작업 단위로 구분하고 여러 Agent 가 상호 소통하면서 문제를 처리해야 할 필요가 있습니다. 이를 Multi-Agent 구조라고 합니다.
- LLM 을 이용한 어플리케이션 구성에서 Multi-Agent 구조를 활용하는 이론은 아래 논문에서 제시되었으며, LangChain 라이브러리에서는 LangGraph 로 구현되었습니다.



# 4. LangGraph?

## LangGraph?

- <https://arxiv.org/abs/2308.08155>



 > cs > arXiv:2308.08155

Search... All fields Search

Help | Advanced Search

Computer Science > Artificial Intelligence

[Submitted on 16 Aug 2023 (v1), last revised 3 Oct 2023 (this version, v2)]

**AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation**

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, Chi Wang

AutoGen is an open-source framework that allows developers to build LLM applications via multiple agents that can converse with each other to accomplish tasks. AutoGen agents are customizable, conversable, and can operate in various modes that employ combinations of LLMs, human inputs, and tools. Using AutoGen, developers can also flexibly define agent interaction behaviors. Both natural language and computer code can be used to program flexible conversation patterns for different applications. AutoGen serves as a generic infrastructure to build diverse applications of various complexities and LLM capacities. Empirical studies demonstrate the effectiveness of the framework in many example applications, with domains ranging from mathematics, coding, question answering, operations research, online decision-making, entertainment, etc.

Comments: 43 pages (10 pages for the main text, 3 pages for references, and 30 pages for appendices)

Subjects: **Artificial Intelligence (cs.AI)**; Computation and Language (cs.CL)

Cite as: [arXiv:2308.08155 \[cs.AI\]](https://arxiv.org/abs/2308.08155)  
(or [arXiv:2308.08155v2 \[cs.AI\]](https://arxiv.org/abs/2308.08155v2) for this version)  
<https://doi.org/10.48550/arXiv.2308.08155>

**Submission history**

From: Qingyun Wu [[view email](#)]  
[v1] Wed, 16 Aug 2023 05:57:52 UTC (2,420 KB)  
[v2] Tue, 3 Oct 2023 20:47:10 UTC (6,271 KB)

**Access Paper:**

- [View PDF](#)
- [TeX Source](#)
- [Other Formats](#)

 [view license](#)

Current browse context:

**cs.AI**

< prev | next >

[new](#) | [recent](#) | [2023-08](#)

Change to browse by:

[cs](#)

[cs.CL](#)

**References & Citations**

- [NASA ADS](#)
- [Google Scholar](#)
- [Semantic Scholar](#)

**Export BibTeX Citation**

**Bookmark**







## 4. LangGraph?

### LangGraph?

- LangGraph 가 Multi-Agent 구현만을 위한 도구는 아닙니다. LLM 을 이용한 코딩 작업을 모듈 단위로 나누어서 연동하는데 사용할 수도 있고, 비교적 큰 LLM 개발 프로젝트를 할 때 구조를 효율적으로 만들기 위한 도구로 사용할 수도 있습니다. 이를 위해 LangGraph 에서는 노드, 엣지, 워크플로우 등의 개념을 사용하여 작업을 모듈화 할 수 있도록 도와줍니다. 또한 사용자가 만든 구조를 시각화 하기 위한 도구도 제공됩니다



## 4. LangGraph?

### LangGraph 의 기능

- 시각화
  - 에이전트와 도구, 데이터 흐름 등을 노드와 에지로 표현하여 시스템의 전체적인 구조를 시각적으로 보여줍니다.
- 관리
  - 에이전트와 도구를 생성, 수정, 삭제하고, 이들 간의 연결 관계를 관리할 수 있습니다.
- 디버깅
  - 시스템에서 발생하는 문제를 찾아내고 해결하는 데 도움을 줍니다.
- 저장 및 로딩
  - 생성된 그래프를 저장하고 다시 로딩하여 재사용할 수 있습니다.



## 4. LangGraph?

### LangGraph 사용 이점

- 복잡성 관리
  - 복잡한 에이전트 시스템을 간단한 그래프로 표현하여 시스템의 이해를 돕습니다.
- 오류 감지
  - 그래프를 통해 데이터 흐름의 문제점이나 불필요한 연결을 쉽게 찾아낼 수 있습니다.
- 시스템 설계
  - 새로운 에이전트를 추가하거나 기존 에이전트를 수정할 때 그래프를 기반으로 시스템 설계를 변경할 수 있습니다.
- 팀 협업
  - 팀원들과 시스템 구조를 공유하고 함께 시스템을 개발할 수 있습니다.

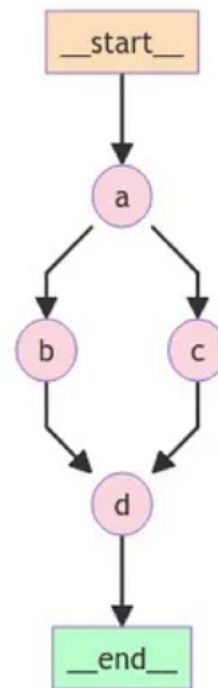


## 4. LangGraph?

### LangGraph 개념

- LangGraph 가 사용하는 노드, 엣지, 빌더(워크플로우), 그래프. LangGraph 의 역할을 단순화하면, 작업을 모듈 단위로 나누어서 관리하고 서로간에 유기적으로 연동해서 문제를 해결할 수 있도록 하는 것

```
builder = StateGraph(State)
builder.add_node("a", ReturnNodeValue("I'm A"))
builder.set_entry_point("a")
builder.add_node("b", ReturnNodeValue("I'm B"))
builder.add_node("c", ReturnNodeValue("I'm C"))
builder.add_node("d", ReturnNodeValue("I'm D"))
builder.add_edge("a", "b")
builder.add_edge("a", "c")
builder.add_edge("b", "d")
builder.add_edge("c", "d")
builder.set_finish_point("d")
graph = builder.compile()
```





## 4. LangGraph?

### LangGraph 개념

- 노드 (Node)
  - 특정 작업을 모듈화 한 구현체로 Agent, 함수 또는 클래스를 말합니다.
  - 따라서 작업이 실행되는 단위이기도 합니다.
  - 위 이미지에서 a, b, c, d 에 해당
- 엣지 (Edge)
  - 각 노드들을 이어주는 라인에 해당하며 노드간 작업의 전환을 의미합니다.
  - 노드가 작업을 마치면 실행해야할 다음 작업을 나타내는데 사용됩니다
  - 정적인 실행(add\_edge), 조건부 실행(add\_conditional\_edges)을 사용할 수 있습니다



## 4. LangGraph?

### LangGraph 개념

- 빌더, 워크플로우 (Builder, Workflow)
  - 위 이미지 좌측 코드에서 빌더로 표시되는 변수
  - 실제로 사용할 그래프의 타입에 따라 만들어지는 그래프 builder 입니다.
  - 노드나 엣지가 빌더에 등록됨으로써 서로 결합하여 실행될 준비가 됩니다
- 그래프 (Graph)
  - 빌더를 컴파일하면 생성되는 인스턴스입니다.
  - `graph.invoke()` 를 통해 그래프를 실행하면 설정한 노드와 엣지의 구조대로 작업을 수행합니다.
- 라우터 (Router)
  - `add_conditional_edges()` 를 통해 조건부로 노드간 연결을 할 때, 어떤 조건에서 어떤 노드가 연결되어야 할지에 대한 코드를 담고있는 함수가 라우터입니다.
  - 위 이미지에는 표시되지 않았지만, 실습 코드에서는 라우터 함수를 정의해서 사용합니다.



## 4. LangGraph?

### LangGraph 주요 유형

- StateGraph
  - 상태 기반 워크플로우를 모델링하는 데 사용
  - 복잡한 의사결정 로직과 조건부 흐름을 구현
- MessageGraph
  - 메시지 기반 시스템을 모델링하는 데 사용
  - 메시지의 흐름과 변환을 추적하는 데 유용
- DAG (Directed Acyclic Graph)
  - 비순환 방향 그래프
  - 작업의 순서와 의존성을 모델링
- Channel
  - 비동기 통신을 위한 단순한 그래프 구조
- Pydantic Graph
  - Pydantic 모델을 사용하여 그래프의 노드와 엣지를 정의
  - 데이터 유효성 검사와 직렬화를 그래프 구조에 통합
- TypedGraph
  - 강력한 타입 검사를 제공하는 그래프 구조
  - 복잡한 시스템에서 타입 안전성을 보장



## 2. 실습

### | 실습 21: 헬로 랭그래프 만들기

```
pip install langgraph==0.6.2 grandalf==0.8
```

```
hello_langgraph.py
```





## 2. 실습

### | 실습 22: 감정 분석 챗봇에 적용해 보기

`conditional_routing.py`

# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약



## 학습목표

- 이 AI 에이전트 프로토콜의 개념과 비즈니스 임팩트를 이해한다
- MCP와 A2A의 기술적 특징과 활용 방안을 구분할 수 있다

## 눈높이 체크

- AI 에이전트 프로토콜 기반 업무 효율화 방안 도출

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## MCP(Model Context Protocol) 핵심 개념

- MCP(Model Context Protocol)는 2024년 11월 25일 Anthropic이 오픈소스로 공개한 개방형 표준으로, 개발자들이 데이터 소스와 AI 기반 도구 간에 안전하고 양방향적인 연결을 구축할 수 있도록 하는 프로토콜

### Introduction

### Get started with the Model Context Protocol (MCP)

MCP is an open protocol that standardizes how applications provide context to large language models (LLMs).

Think of MCP like a USB-C port for AI applications



**Growing list of pre-built integrations**

LLM can directly plug into



**Standardized way to build custom integrations**

For AI applications



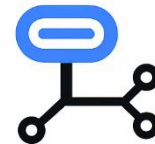
**Open protocol free to implement and use**

Open protocol free to implement and use



**Flexibility to change between apps**

And take your context with you



A standard interface  
for connecting LLMs  
with the world

## MCP(Model Context Protocol) 핵심 개념

### ● 산업 현황과 필요성

#### ◦ 기존 AI 시스템의 한계:

- 정보 사일로와 레거시 시스템에 갇힌 AI
- 각 데이터 소스마다 맞춤형 구현 필요
- $N \times M$  데이터 통합 문제 ( $N$ 개 AI  $\times$   $M$ 개 데이터 소스)
- 확장 가능한 연결 시스템 구축 어려움

#### ◦ MCP의 해결 방안:

- 단일 프로토콜로 모든 데이터 소스 연결
- 파편화된 통합을 범용 표준으로 대체
- 프런티어 모델들의 더 나은 응답 생성 지원
- 개발 시간과 비용의 대폭 절감

# 1. AI 에이전트 프로토콜 - 클로드 MCP

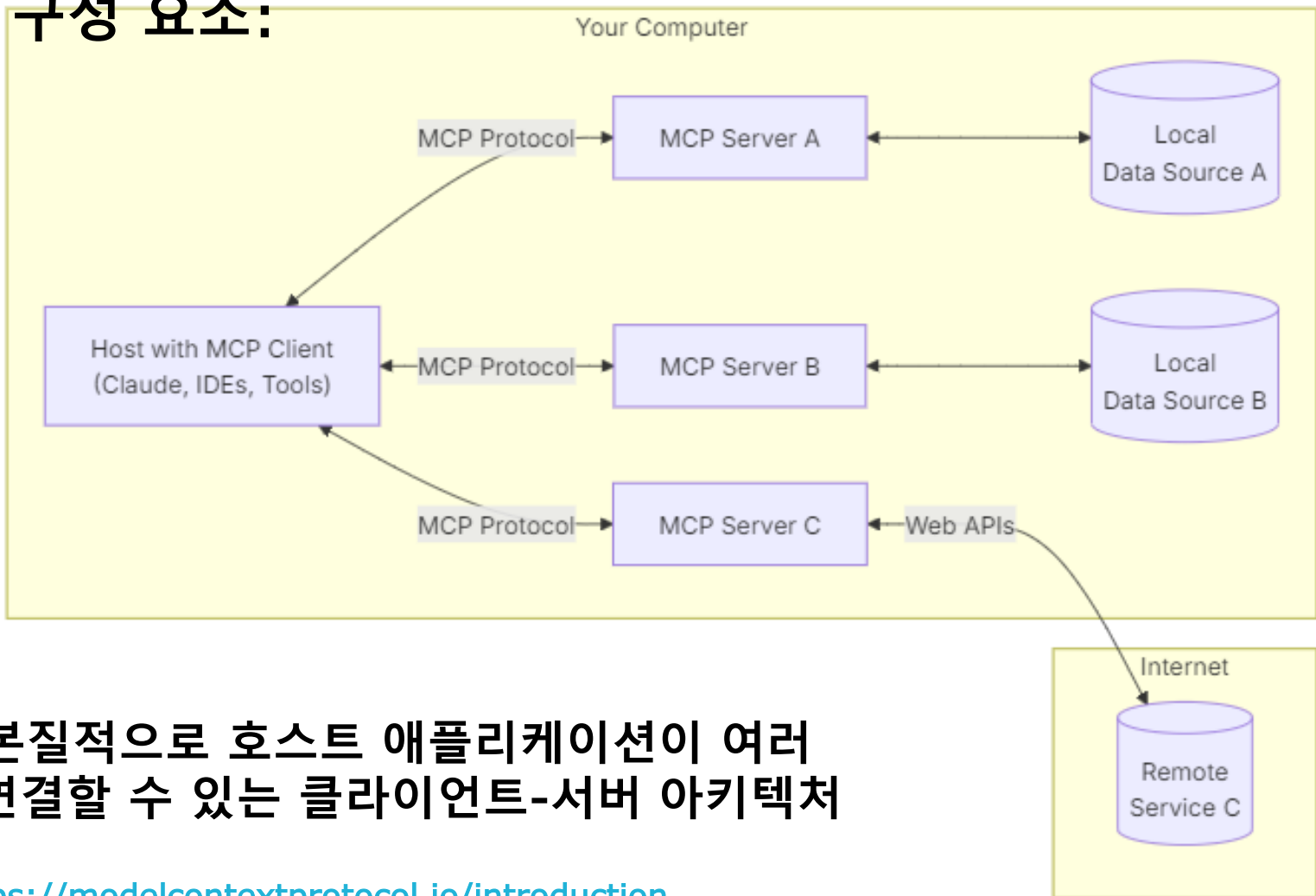
## | MCP 기술 아키텍처 심화 분석

- MCP 서버 (Data Provider) 구조
  - MCP 서버는 개발자가 MCP 서버를 통해 데이터를 노출하거나, 이러한 서버에 연결하는 AI 애플리케이션(MCP 클라이언트)을 구축할 수 있는 구조 Introducing the Model Context Protocol \ Anthropic를 제공합니다.

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## MCP 기술 아키텍처 심화 분석

- 주요 구성 요소:



MCP는 본질적으로 호스트 애플리케이션이 여러 서버에 연결할 수 있는 클라이언트-서버 아키텍처

## MCP 기술 아키텍처 심화 분석

- MCP 호스트 :
  - MCP를 통해 데이터에 액세스하려는 Claude Desktop, IDE 또는 AI 도구와 같은 프로그램
- MCP 클라이언트 :
  - 서버와 1:1 연결을 유지하는 프로토콜 클라이언트
- MCP 서버 :
  - 표준화된 모델 컨텍스트 프로토콜을 통해 각각 특정 기능을 노출하는 경량 프로그램
- 로컬 데이터 소스 :
  - MCP 서버가 안전하게 액세스할 수 있는 컴퓨터의 파일, 데이터베이스 및 서비스
- 원격 서비스 :
  - MCP 서버가 연결할 수 있는 인터넷(예: API를 통해)을 통해 사용할 수 있는 외부 시스템



# 1. AI 에이전트 프로토콜 - 클로드 MCP

## MCP 기술 아키텍처 심화 분석

# MCP 서버 기본 구조 예시

```
class MCPServer:
```

```
 def __init__(self):
```

```
 self.resources = {} # 데이터 리소스 관리
```

```
 self.tools = {} # 실행 가능한 도구들
```

```
 self.prompts = {} # 준비된 프롬프트 템플릿
```

```
 def add_resource(self, name, config):
```

```
 """구조화된 데이터 추가"""
```

```
 self.resources[name] = {
```

```
 "type": "document_snippet|code_fragment|database_record",
```

```
 "content": config["data"],
```

```
 "permissions": config.get("permissions", ["read"]),
```

```
 "metadata": config.get("metadata", {})
```

```
 }
```

```
 def add_tool(self, name, function):
```

```
 """실행 가능한 함수 추가"""
```

```
 self.tools[name] = {
```

```
 "function": function,
```

```
 "description": function.__doc__,
```

```
 "parameters": self._extract_parameters(function)
```

```
 }
```

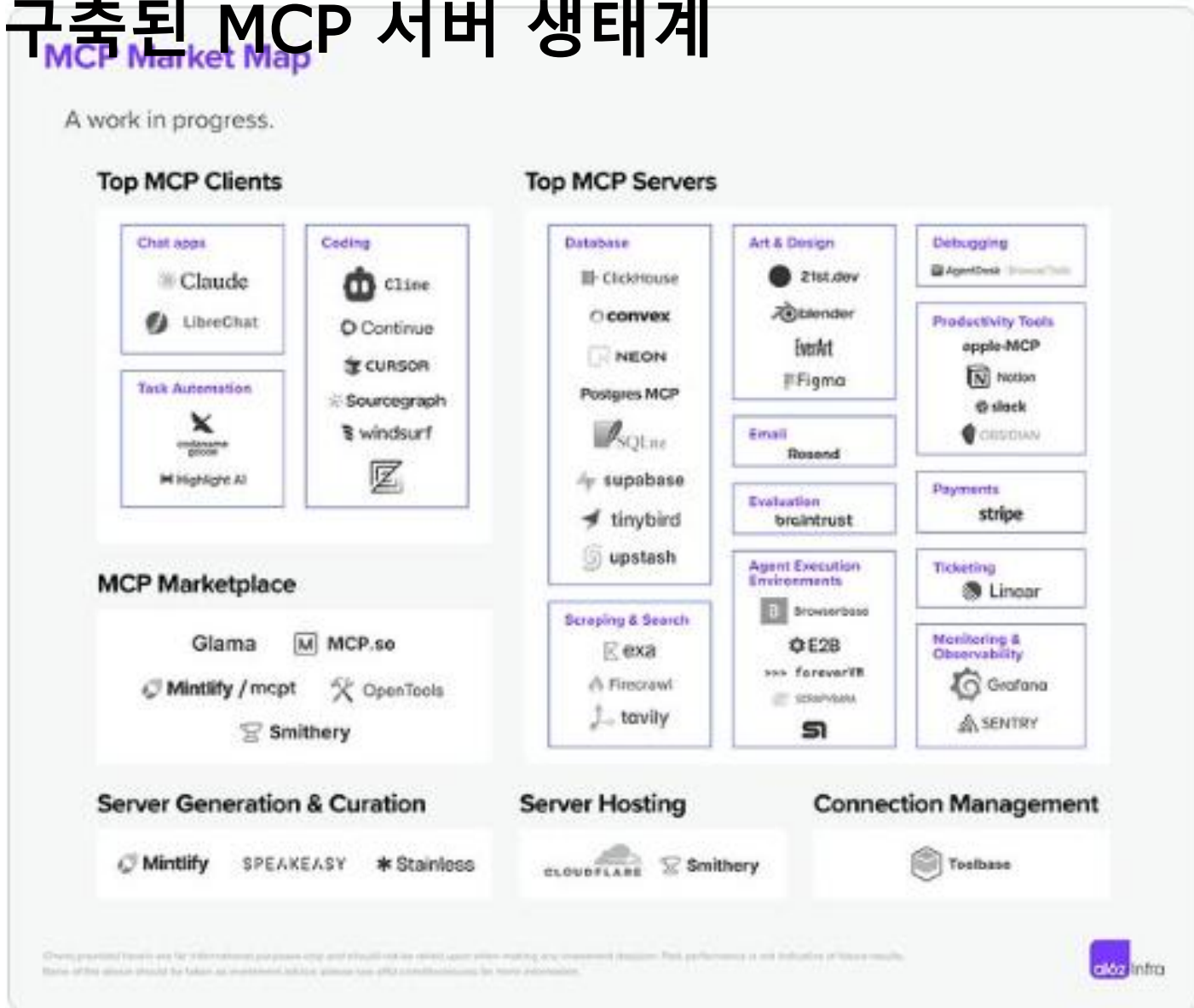
# 1. AI 에이전트 프로토콜 - 클로드 MCP

## | MCP 기술 아키텍처 심화 분석

- 사전 구축된 MCP 서버 생태계
  - Anthropic은 Google Drive, Slack, GitHub, Git, Postgres, Puppeteer 등 인기 있는 엔터프라이즈 시스템을 위한 사전 구축된 MCP 서버들을 공유하고 있습니다

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## ● 사전 구축된 MCP 서버 생태계



# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

### Cursor Settings

CursorTest1.py

Cursor Settings X



onlookertozip@gmail.com  
Pro Trial



General



Chat



Tab



Models



Rules



Indexing & Docs



MCP Tools



Network



Beta



Docs

#### MCP Tools

##### No MCP Tools

Add a custom MCP tool here or configure project-specific tools in  
<project-root>/cursor/mcp.json

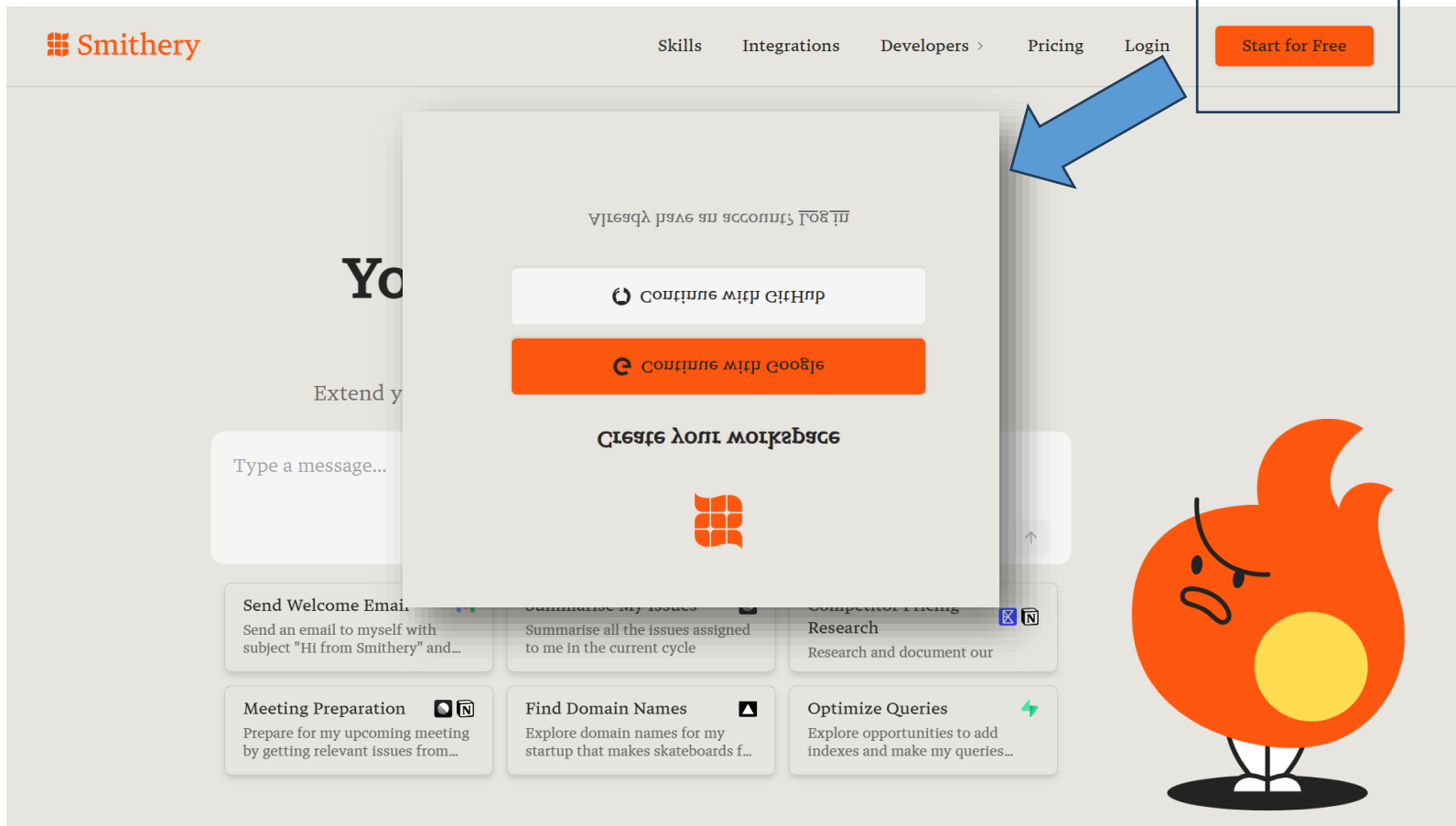
Add Custom MCP

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

### MCP 설치

- <https://smithery.ai/>



The screenshot shows the Smithery website interface. At the top, there is a navigation bar with links for Skills, Integrations, Developers, Pricing, and Login. A prominent orange button labeled "Start for Free" is located in the top right corner, enclosed in a black rectangular box. A large blue arrow points from this button towards the center of the page. In the center, a modal window is displayed with the text "Already have an account? Log in" at the top. Below this text are two buttons: "Continue with GitHub" (white with a black border) and "Continue with Google" (solid orange). Below these buttons is a section titled "Create your workspace" with the Smithery logo (a 3x3 grid of orange squares) underneath. At the bottom of the page, there is a grid of six cards, each representing a different AI agent or tool, such as "Send Welcome Email", "Summarize my issues", "Research", "Meeting Preparation", "Find Domain Names", and "Optimize Queries". To the right of the main content area, there is a large, stylized orange flame character with a yellow circle in the center and a simple face.

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

### ● 문제 상황 예시

- 처음엔 Sequential Thinking을 활용하지 않고 일반적인 프롬프트만 사용했는데, AI의 응답이 일관성이 없고 중간 과정에서 논리가 꼬이는 경우가 많았다. 특히 복잡한 문제에서는 단계별 검증 없이 결론으로 건너뛰는 경우가 빈번했다.

### ● 해결

- Sequential Thinking MCP Server를 활용하여 "sequential thinking tool을 사용해서(Use Sequential Thinking MCP)"라는 명령어를 프롬프트에 추가했다. 이를 통해 AI가 체계적으로 문제를 분석하고, 각 단계를 검증하며, 필요시 이전 단계로 돌아가 수정하는 구조화된 사고 과정을 거치게 됐다.

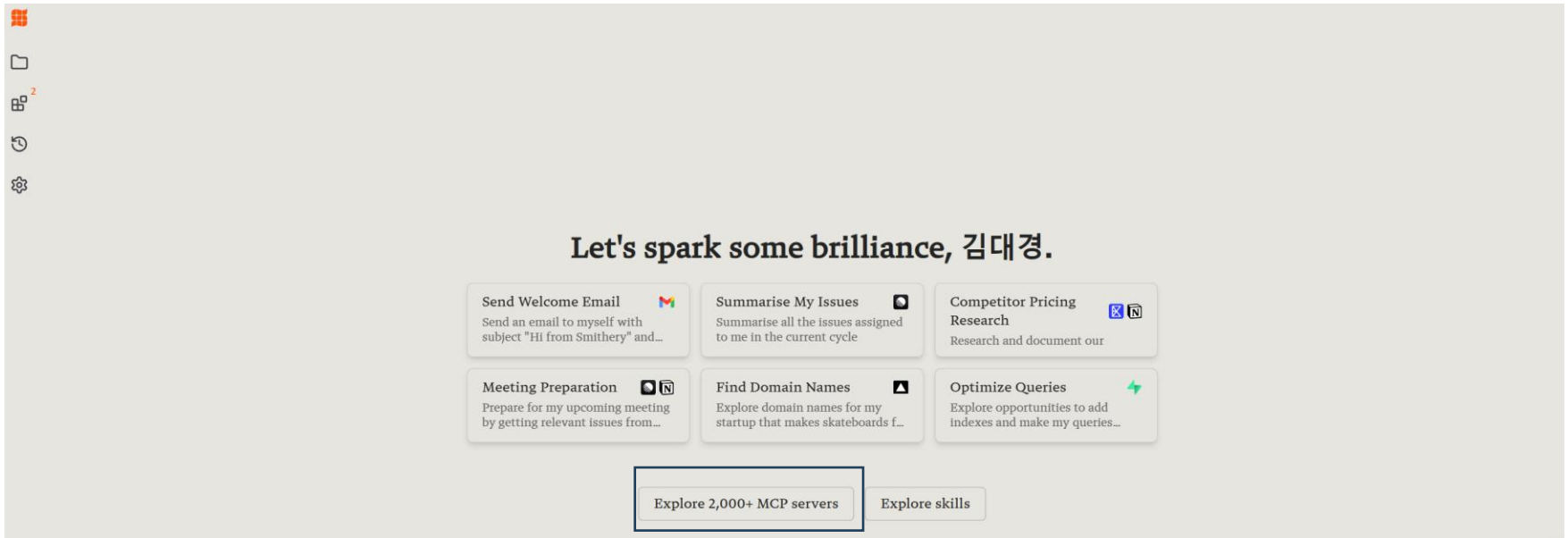
### ● ex) 질의 예시

- Sequential Thinking MCP를 사용해서 다음 내용에 대해 대답해줘.
- 간단한 TODO 리스트 기능을 web페이지 형식으로 구현하려고 해. 어떤 기술 스택으로 구현 하는게 좋을까?

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

### Github MCP Server 이용해 보기



# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

Github MCP Server 이용해 보기 > Sequential Thinking

The screenshot displays the Cursor.AI MCP Server marketplace interface. At the top, a search bar contains the text "Sequential Thinking". Below the search bar, a dropdown menu titled "SUGGESTIONS" lists several servers:

- Sequential Thinking** (@smithery-ai/server-sequential-thinking · 134,298 uses)
- Sequential Thinking** (@kiennd/reference-servers · 15,908 uses)
- Pete Thinking Server** (@PewterZz/sequential-thinking-mcp · 0 uses)
- mcp-server-mas-sequential-thinkingfork** (@renCosta2025/mcp-server-mas-sequential-thinkingfork · 0 uses)

Below the suggestions, a grid of server cards is visible, including:

- Canva**: Canva offers a drag-and-drop design suite for creating social media graphics, presentations, and marketing...
- Google Drive**: Google Drive is a cloud storage solution for uploading, sharing, and collaborating on files across devices, with...
- Exa Search**: Fast, intelligent web search and web crawling. New mcp tool: Exa-code is a context tool for coding agents. It...
- Slack**: Slack is a channel-based messaging platform. With Slack, people can work together more effectively, connect all...
- Reddit**: Reddit is a social news platform with user-driven...
- Google Calendar**: Google Calendar is a time management tool providing...
- Gmail**: Manage Gmail end-to-end: send, draft, reply, forward...

The interface also includes a left sidebar with filters (Filter by, Status, Location, Ownership, Advanced) and a top navigation bar with links to Skills, Integrations, Developers, Pricing, Chat, and a Personal dropdown menu.



# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

### Github MCP Server 이용해 보기 > Sequential Thinking

The screenshot displays the Cursor.AI MCP Server marketplace interface. At the top, a search bar contains the text "Sequential Thinking". The interface is divided into a left sidebar with filters and a main grid of server cards.

**Left Sidebar Filters:**

- Filter by**
- Status**
  - Official
- Location**
  - Remote
  - Local
- Ownership**
  - My servers
- Advanced**
  - Owner
  - Repository
- Categories**
  - Memory
  - Academic Research
  - Agentic Tools
  - Developer Tools
  - Web Search

**Main Grid of MCP Servers:**

Name	Username	Status	Location	Stars
Sequential Thinking	@smithery-ai/server-sequential-thinking	S	Remote	134.29k
Sequential Thinking	@kiennd/reference-servers		Local	15.90k
Pete Thinking Server	@PewterZz/sequential-thinking-mcp	P	Remote	
sequentialthinking	@vmanoilov/sequentialthinking	S		
sequentialthinking2	@vmanoilov/sequentialthinking2	S		
sequentialthinking4	@vmanoilov/sequentialthinking4	S		
mcp-sequentialthinking-tools	@code-craka/mcp-sequentialthinking-tools	M		
think	@vmanoilov/think	T		
mcp-server-mas-sequential-thinkingfork	@renCosta2025/mcp-server-mas-sequential-think...	M		

At the bottom of the grid, there is a message: "Looking for an MCP that doesn't exist yet? [Request it on GitHub](#)".

The bottom navigation bar shows "< Previous", a highlighted "1", and "Next >".

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

Auto > Cursor 선택

The screenshot displays the Cursor AI interface for configuring an MCP server. At the top, there's a navigation bar with links for Skills, Integrations, Developers, Pricing, Chat, and a user profile. The main content area shows the 'Sequential Thinking' server by @smithery-ai, which was last deployed 2 months ago. It has 134.3k calls and is open source. A 'Run' button is visible. Below this, there's an 'About' section describing it as an MCP server implementation for dynamic and reflective problem-solving. The 'Capabilities' section shows one tool, 'sequentialthinking', which is a detailed tool for dynamic and reflective problem-solving through thoughts. On the right, there's a 'Connect' section with a 'Configure' button and a text box for the connection URL: 'https://server.smithery.ai/@smithery-ai/server-sequential-thi'. Below this, there's a section 'Or add to your client' with tabs for Auto, JSON, TypeScript, and Python. The 'Auto' tab is selected, and it shows a list of clients: ChatGPT, Poke, Claude Desktop, Claude Code, Cursor, Qordinate, and Codex.

Search servers...

Skills Integrations Developers > Pricing Chat 대경 / Personal v

**Sequential Thinking**  
@smithery-ai/server-sequential-thinking • last deployed 2 months ago

Remote Quick Setup 134.3k calls Open Source

**About**  
An MCP server implementation that provides a tool for dynamic and reflective problem-solving through a structured thinking process.

**+ Capabilities**  
Tools 1

**sequentialthinking**  
A detailed tool for dynamic and reflective problem-solving through thoughts. This tool helps analyze problems through a flexible thinking process that can adapt and evolve. Each thought can build on, question, or revise previous insights as understanding deepens.  
When to use this tool:

**Connect** Configure

Get connection URL ⓘ  
https://server.smithery.ai/@smithery-ai/server-sequential-thi  
Client doesn't support OAuth yet or link isn't working? [Get URL with keys instead](#)

Or add to your client

Auto JSON TypeScript Python

Search clients

- ChatGPT
- Poke
- Claude Desktop
- Claude Code
- Cursor
- Qordinate
- Codex

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기 One-Click Install

The screenshot displays the Cursor AI interface for the 'Sequential Thinking' MCP application. The application is by '@smithery-ai/server-sequential-thinking' and was last deployed 2 months ago. It has 134.3k calls and is open source. The interface includes a 'Run' button and a 'Configure' button. A modal window is open, asking 'Cursor을(를) 여시겠습니까?' (Do you want to use Cursor?). It provides instructions to open the application from 'https://smithery.ai' and offers a checkbox for '항상 smithery.ai에서 연결된 앱에 있는 이 유형의 링크를 열도록 허용' (Allow opening links of this type in apps connected to smithery.ai). The modal has two buttons: 'Cursor 열기' (Open Cursor) and '취소' (Cancel). A blue arrow points from the 'One-Click Install' button in the background to the 'Cursor 열기' button in the modal. The background interface also shows a 'Get connection URL' section with a URL 'https://server.smithery.ai/@smithery-ai/server-sequential-thin' and a 'Connect this server to Cursor with one click' button labeled 'One-Click Install ->'. Below this, it says 'Install this server using the terminal command below:' and provides a terminal command: 'npx -y @smithery/cli@latest install @smithery-ai/server-sequential-th'. A warning at the bottom states: 'Your smithery key is sensitive. Please don't share it with anyone.'

Sequential Thinking  
@smithery-ai/server-sequential-thinking • last deployed 2 months ago

Remote Quick Setup 134.3k calls Open Source

Run ▶

Cursor을(를) 여시겠습니까?

https://smithery.ai에서 이 애플리케이션을 열려고 합니다.

☐ 항상 smithery.ai에서 연결된 앱에 있는 이 유형의 링크를 열도록 허용

Cursor 열기 취소

Get connection URL ⓘ

smithery.ai/@smithery-ai/server-sequential-thin

Or add to your client

JSON TypeScript Python

Cursor

Connect to this remote server using its HTTP URL:

https://server.smithery.ai/@smithery-ai/server-sequential-thin

Connect this server to Cursor with one click.

One-Click Install →

Install this server using the terminal command below:

```
terminal
npx -y @smithery/cli@latest install @smithery-ai/server-sequential-th
```

Your smithery key is sensitive. Please don't share it with anyone.

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

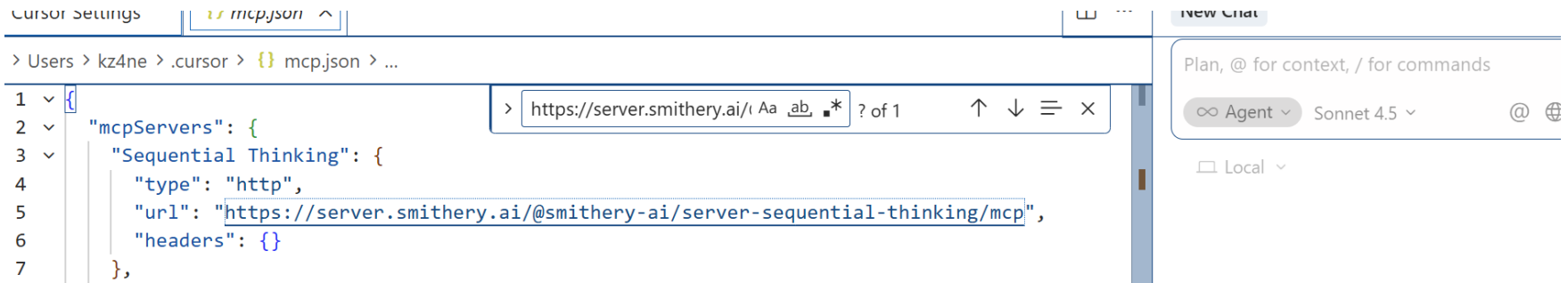
### Install

The screenshot displays the Cursor AI application interface. On the left, a sidebar shows the user profile 'kz4network@gmail.com' and a list of settings categories: General, Agents, Tab, Models, Cloud Agents, Tools & MCP (selected), Rules, Memories, Comm..., Indexing & Docs, Network, Beta, and Docs. The main panel is titled 'Cursor Settings' and features a 'Tools' section with 'Browser Automation' (Ready (Chrome detected)). Below this, the 'Installed MCP Servers' section shows a form to 'Install MCP Server?'. The form fields are: Name (Sequential Thinking), Type (streamableHttp), and URL (https://server.smithery.ai/@smithery-ai/server-sequential-tl). There are 'Cancel' and 'Install' buttons. Below the form, a list of installed servers is shown: 'Sequential Thinking' (1 tools enabled) and 'supabase-mcp-lite' (4 tools enabled), both with toggle switches turned on. A '+ New MCP Server' button is at the bottom. On the right, a 'New Chat' window is visible, showing the plan '@ Sonnet 4.5' and a 'Local' dropdown.

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

### Install 결과



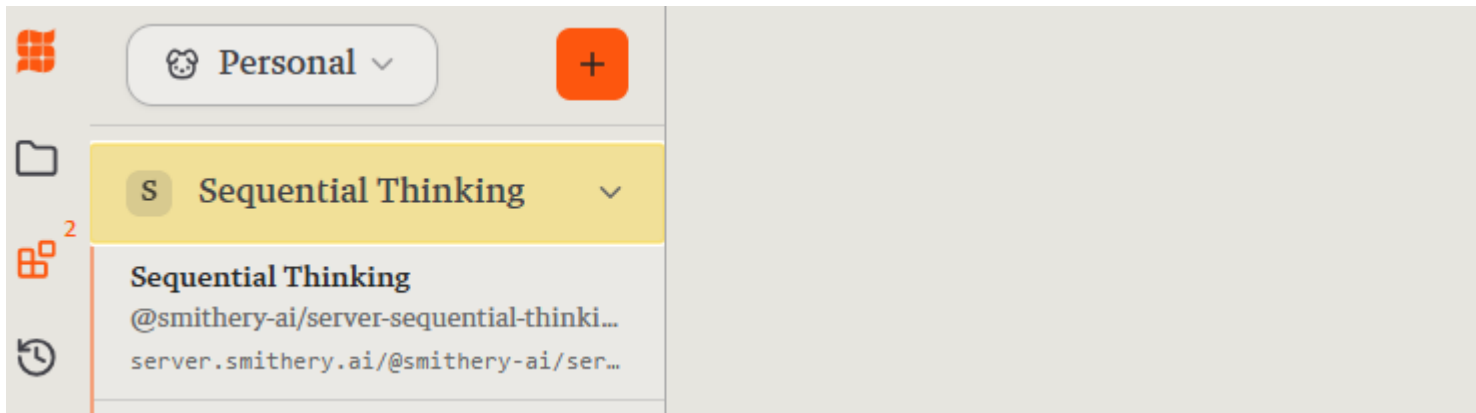
The screenshot shows the Cursor AI settings interface. The left sidebar displays the file explorer path: > Users > kz4ne > .cursor > {} mcp.json > ... The main editor area shows the contents of the mcp.json file, which is a JSON configuration for MCP servers. The configuration includes a section for "mcpServers" with a sub-section for "Sequential Thinking". The "url" field is set to "https://server.smithery.ai/@smithery-ai/server-sequential-thinking/mcp". A search bar at the top of the editor shows the text "https://server.smithery.ai/". The right sidebar shows the "New Chat" button and the current chat context, which includes "Plan, @ for context, / for commands", "Agent", "Sonnet 4.5", and "Local".

```
1 {
2 "mcpServers": {
3 "Sequential Thinking": {
4 "type": "http",
5 "url": "https://server.smithery.ai/@smithery-ai/server-sequential-thinking/mcp",
6 "headers": {}
7 },
8 }
9 }
```

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## | Cursor.AI 로 MCP 사용하기

### Install 결과



# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

```
{
 "mcpServers": {
 "Sequential Thinking": {
 "type": "http",
 "url": "https://server.smithery.ai/@smithery-ai/server-sequential-thinking/mcp",
 "headers": {}
 },
 "supabase-mcp-lite": {
 "type": "http",
 "url": "https://server.smithery.ai/@pinion05/supabase-mcp-lite/mcp",
 "headers": {}
 }
 }
}
```



The screenshot shows a code editor window with the file path `C: > Users > kz4ne > .cursor > {} mcp.json > ...`. The editor displays the following JSON configuration for MCP servers:

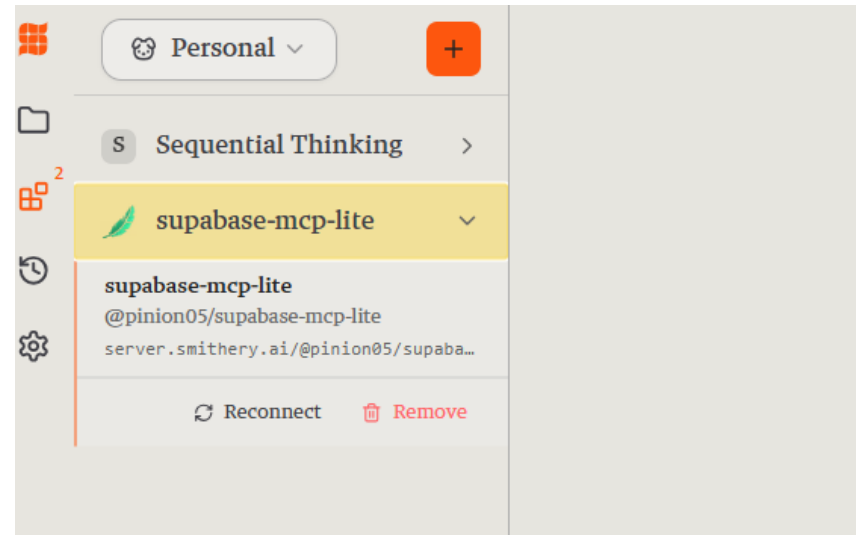
```
1 {
2 "mcpServers": {
3 "Sequential Thinking": {
4 "type": "http",
5 "url": "https://server.smithery.ai/@smithery-ai/server-sequential-thinking/mcp",
6 "headers": {}
7 },
8 "supabase-mcp-lite": {
9 "type": "http",
10 "url": "https://server.smithery.ai/@pinion05/supabase-mcp-lite/mcp",
11 "headers": {}
12 }
13 }
14 }
```

A search bar at the top right of the editor shows the search term `> https://server.smithery.ai/` with a dropdown menu containing `Aa`, `ab`, and `*`. The search results indicate `? of 1` matches.

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

```
{
 "mcpServers": {
 "Sequential Thinking": {
 "type": "http",
 "url": "https://server.smithery.ai/@smithery-ai/server-sequential-thinking/mcp",
 "headers": {}
 },
 "supabase-mcp-lite": {
 "type": "http",
 "url": "https://server.smithery.ai/@pinion05/supabase-mcp-lite/mcp",
 "headers": {}
 }
 }
}
```

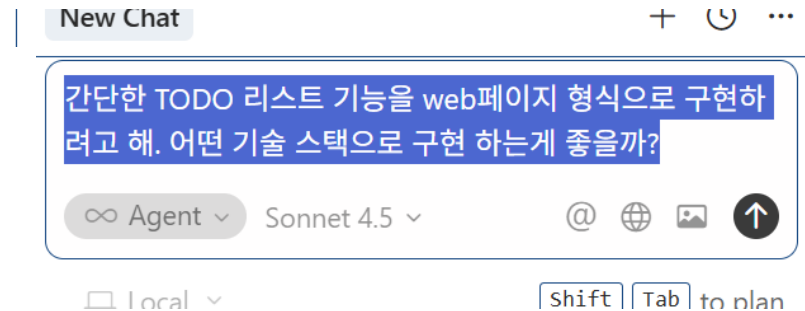




# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

간단한 TODO 리스트 기능을 web페이지 형식으로 구현하려고 해. 어떤 기술 스택으로 구현 하는게 좋을까?



# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

Sequential Thinking MCP를 사용해서 다음 내용에 대해 대답해줘.  
간단한 TODO 리스트 기능을 web페이지 형식으로 구현하려고 해. 어떤 기술 스택으로 구현 하는게 좋을까?

Sequential Thinking MCP를 사용해서 다음 내용에 대해 대답해줘.

간단한 TODO 리스트 기능을 web페이지 형식으로 구현하려고 해. 어떤 기술 스택으로 구현 하는게 좋을까?

∞ Agent ▾

Sonnet 4.5 ▾



# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

Sequential Thinking 을 진행한 후 최종 답변을 생성하는 것을 볼 수 있습니다.

Sequential Thinking MCP를 사용해서 다음 내용에 대해 대답해줘.

간단한 TODO 리스트 기능을 web페이지 형식으로 구현하려고 해. 어떤 기술 스택으로 구현 하는게 좋을까?

Thought for 2s

Sequential Thinking을 활용하여 TODO 리스트 웹페이지를 위한 최적의 기술 스택을 분석해드리겠습니다.

✓ Ran sequentialthinking

Thought for 2s

✓ Ran sequentialthinking

Thought for 1s

✓ Ran sequentialthinking

Thought for 1s

✓ Ran sequentialthinking

Thought for 1s

✓ Ran sequentialthinking

Use Allowlist ▾

Thought for 2s

체계적인 분석을 완료했습니다! TODO 리스트 웹페이지를 위한 최적의 기술 스택을 추천드리겠습니다.

### 🎯 최종 추천 기술 스택

Next.js 14 (App Router) + TypeScript + Tailwind CSS

### 📊 분석 과정

#### 요구사항 분석

- "간단한" TODO 리스트: 추가, 완료, 삭제 기능
- 웹페이지 형식
- 워크스페이스 이름 `Nextjs_base` → Next.js 고려 중인 것으로 판단

#### 기술 스택 비교

기술 스택	장점	단점	적합도

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

<https://github.com/modelcontextprotocol/servers/tree/main/src/sequentialthinking>

The screenshot displays the Cursor.AI IDE interface. On the left, the 'Files' sidebar shows a project structure with folders like .github, .vscode, scripts, src, and sequentialthinking. The 'sequentialthinking' folder is selected. The main editor area shows the 'servers / src / sequentialthinking /' directory. It contains a bullet point: '• Generate and verify solution hypotheses'. Below this, the 'Tool' section is titled 'sequential\_thinking'. A description follows: 'Facilitates a detailed, step-by-step thinking process for problem-solving and analysis.' Under the 'Inputs:' section, a list of parameters is provided:

- `thought` (string): The current thinking step
- `nextThoughtNeeded` (boolean): Whether another thought step is needed
- `thoughtNumber` (integer): Current thought number
- `totalThoughts` (integer): Estimated total thoughts needed
- `isRevision` (boolean, optional): Whether this revises previous thinking
- `revisesThought` (integer, optional): Which thought is being reconsidered
- `branchFromThought` (integer, optional): Branching point thought number
- `branchId` (string, optional): Branch identifier
- `needsMoreThoughts` (boolean, optional): If more thoughts are needed

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

thought (string): The current thinking step

nextThoughtNeeded (boolean): Whether another thought step is needed

thoughtNumber (integer): Current thought number

totalThoughts (integer): Estimated total thoughts needed

isRevision (boolean, optional): Whether this revises previous thinking

revisesThought (integer, optional): Which thought is being reconsidered

branchFromThought (integer, optional): Branching point thought number

branchId (string, optional): Branch identifier

needsMoreThoughts (boolean, optional): If more thoughts are needed

thought(문자열) : 현재 생각 단계

nextThoughtNeeded (boolean) : 다른 생각 단계가 필요한지 여부

thoughtNumber (정수) : 현재 생각 번호

totalThoughts (정수) : 필요한 예상 총 생각 수

isRevision (부울, 선택 사항) : 이전 생각을 수정할지 여부

revisesThought (정수, 선택 사항) : 재검토 중인 생각

branchFromThought (정수, 선택 사항) : 분기점 생각 번호

branchId (문자열, 선택 사항) : 브랜치 식별자

needsMoreThoughts (boolean, 선택 사항) : 더 많은 생각이 필요한 경우

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

'thought': '간단한 TODO 리스트 웹페이지를 구현하기 위한 기술 스택을 추천해야 한다. 사용자가 \"간단한\" 이라고 명시했으므로, 복잡하지 않고 빠르게 구현할 수 있는 기술들을 고려해야 한다.'

먼저 TODO 리스트의 기본 기능들을 생각해보자:

- 할 일 추가
- 할 일 완료 체크/해제
- 할 일 삭제
- 할 일 목록 표시
- 데이터 저장 (로컬 스토리지 또는 서버)

이제 각 계층별로 기술 스택을 고려해보자.',

'thoughtNumber': 1,  
'totalThoughts': 5,  
'nextThoughtNeeded': true

한 기술 스택을 추천해야 한다. 사용자가 \"간단한\" 이라고 명시했으므로, 복잡하지 않고 빠르게 구현할 수 있는 기술들을 고려해야 한다.

먼저 TODO 리스트의 기본 기능들을 생각해보자:

- 할 일 추가
- 할 일 완료 체크/해제
- 할 일 삭제
- 할 일 목록 표시
- 데이터 저장 (로컬 스토리지 또는 서버)

이제 각 계층별로 기술 스택을 고려해보자.',

# 1. AI 에이전트 프로토콜 - 클로드 MCP

## Cursor.AI 로 MCP 사용하기

### 총 6번 Sequential Thinking이 진행

'thought': '간단한 TODO 리스트 웹페이지를 구현하기 위한 기술 스택을 추천해야 한다. 사용자가 \"간단한\" 이라고 명시했으므로, 복잡하지 않고 빠르게 구현할 수 있는 기술들을 고려해야 한다.'

Thought for 6s

제시하신 사고 과정을 이어서 진행하겠습니다!

✓ Ran sequentialthinking

Thought for 2s

✓ Ran sequentialthinking

Thought for 1s

✓ Ran sequentialthinking

Thought for 2s

✓ Ran sequentialthinking

Thought for 1s

✓ Ran sequentialthinking

Use Allowlist

Thought for 2s

#### 체계적 분석 완료

Sequential Thinking을 통해 단계별로 분석한 결과를 정리해드립니다.



## 2. 실습

### | 실습 23: MCP 서버 만들기

Node.js® 다운로드

FastMCP

`simple_mcp_server.py`

`simple_mcp_client.py`



## Node.js 설치

- Node.js는 크롬 V8 엔진 위에서 동작하는 자바스크립트 실행 환경
- 설치 확인 방법:
  - 먼저 내 컴퓨터에 Node.js가 설치되어 있는지 확인해 봅시다.
  - MacOS: 터미널 열고 → `node --version`
  - Windows: Win + R → cmd 입력 후 실행창에서 `node --version`
- 버전 번호가 정상적으로 출력되면 이미 설치가 잘 되어 있는 겁니다
- 만약 `command not found` 또는 `'node' is not recognized` 같은 메시지가 뜬다면, Node.js를 새로 설치해야 합니다.
- 운영체제에 맞는 설치 파일을 받으세요.
- 추천 버전: 최신 LTS(Long-Term Support) 버전



### 3. AI 에이전트 프로토콜 - 구글 A2A



#### | 구글 A2A 무엇인가?

- A2A의 정의와 비전

- 2025년 4월 9일 Google이 발표한 Agent2Agent(A2A)는 AI 에이전트 간 통신, 안전한 정보 교환, 다양한 엔터프라이즈 플랫폼이나 애플리케이션 위에서의 활동 조율을 가능하게 하는 새로운 오픈 프로토콜  
Announcing the Agent2Agent Protocol (A2A) - Google Developers Blog입니다.



# 3. AI 에이전트 프로토콜 - 구글 A2A

## 구글 A2A 무엇인가?

- 정의: 특정 목표를 달성하기 위해 환경을 감지(Perceive)하고, 판단(Think)하며, 자율적으로 행동(Act)하는 소프트웨어 개체입니다.
- 핵심 3요소:
  - 센서(Sensor): 주변 환경으로부터 데이터를 수집합니다. (예: 사용자 입력, API 데이터, 센서 값)
  - 처리(Processor/Brain): 수집된 데이터를 바탕으로 최적의 행동을 결정합니다. (예: LLM, 머신러닝 모델)
  - 액추에이터(Actuator): 결정된 행동을 실행하여 환경에 영향을 줍니다. (예: API 호출, 메시지 전송, 기기 제어)
- 예시:
  - 단순한 에이전트: 스팸 메일 필터, 챗봇
  - 복잡한 에이전트: 자율 주행 자동차, 스마트 홈 비서, 자동화된 금융 거래 시스템



### 3. AI 에이전트 프로토콜 - 구글 A2A



#### 왜 지금 AI 에이전트가 중요한가?

- 복잡한 문제 해결: 인간의 개입 없이 여러 단계의 복잡한 작업을 자율적으로 처리할 수 있습니다.
- 초개인화 서비스: 사용자의 맥락과 의도를 깊이 이해하여 맞춤형 서비스를 제공합니다.
- 생산성 혁신: 반복적이고 시간이 많이 소요되는 업무를 자동화하여 인간이 더 창의적인 일에 집중할 수 있도록 돕니다.



### 3. AI 에이전트 프로토콜 - 구글 A2A



## A2A(Agent-to-Agent) 프로토콜 심화 개념

- 산업 파트너십과 생태계

- A2A는 Atlassian, Box, Cohere, Intuit, Langchain, MongoDB, PayPal, Salesforce, SAP, ServiceNow, UKG, Workday 등 50개 이상의 기술 파트너와 Accenture, BCG, Capgemini, Cognizant, Deloitte, HCLTech, Infosys, KPMG, McKinsey, PwC, TCS, Wipro 등 주요 서비스 제공업체의 지원과 기여를 받고 있습니다



### 3. AI 에이전트 프로토콜 - 구글 A2A



#### A2A 핵심 설계 원칙

- 5가지 핵심 원칙:

- A2A 프로토콜 설계 시 5가지 핵심 원칙을 준수했습니다: 1) 에이전트 역량 수용, 2) 기존 표준 기반 구축, 3) 기본 보안, 4) 장기 실행 작업 지원, 5) 협업 가능

- 에이전트 능력 중심 설계

- 기존 도구 중심 vs A2A 에이전트 중심:
- 기존: 에이전트를 단순한 "도구"로 제한
- A2A: 메모리, 도구, 컨텍스트를 공유하지 않아도 자연스럽게 구조화되지 않은 방식으로 협업 가능



### 3. AI 에이전트 프로토콜 - 구글 A2A



#### A2A 핵심 설계 원칙

- **표준 기술 스택 활용**

- A2A는 HTTP, SSE(Server-Sent Events), JSON-RPC 등 기존의 인기 있는 표준 위에 구축되어 기업이 이미 매일 사용하는 기존 IT 스택과 통합하기 쉽습니다. JSON-RPC 2.0을 HTTP(S) 위에서 핵심 통신 방법으로 사용합니다 Google DevelopersTowards Data Science.

- **엔터프라이즈급 보안**

- A2A는 런치 시 OpenAPI의 인증 스킴과 동등한 엔터프라이즈급 인증 및 인가를 지원하도록 설계되었습니다 Announcing the Agent2Agent Protocol (A2A) - Google Developers Blog.

## | Agent Card (에이전트 카드)

- 에이전트들은 JSON 형식의 "Agent Card"를 사용하여 자신의 능력을 광고할 수 있어, 클라이언트 에이전트가 작업을 수행할 수 있는 최고의 에이전트를 식별하고 A2A를 활용해 원격 에이전트와 통신할 수 있습니다



## 1) Agent Card (에이전트 카드)

```
{
 "agent_card": {
 "name": "Financial Analysis Agent",
 "version": "2.1.0",
 "description": "전문적인 재무 분석 및 예측 서비스",
 "capabilities": {
 "skills": [
 "financial_modeling",
 "risk_assessment",
 "market_analysis",
 "budget_planning"
],
 "input_modalities": ["text", "spreadsheet", "json"],
 "output_modalities": ["text", "charts", "reports", "forms"]
 },
 "endpoints": {
 "base_url": "https://finance-agent.company.com",
 "health_check": "/health",
 "capabilities": "/capabilities"
 },
 "authentication": {
 "scheme": "bearer_token",
 "required": true
 },
 "sla": {
 "response_time": "< 30s",
 "availability": "99.9%"
 }
 }
}
```

## Task Management (작업 관리)

- 클라이언트와 원격 에이전트 간의 통신은 작업 완료 지향적이며, 에이전트들이 최종 사용자 요청을 이행하기 위해 작업합니다. 프로토콜에 의해 정의된 "작업" 객체는 생명주기를 가지며, 즉시 완료되거나 장기 실행 작업의 경우 각 에이전트가 작업 완료의 최신 상태에 대해 서로 동기화를 유지할 수 있습니다 [Announcing the Agent2Agent Protocol \(A2A\)](#) - Google Developers Blog.

## Task Management (작업 관리)

- 작업 생명주기:

- 작업 상태 전이:

- submitted → accepted → in\_progress → completed/failed/cancelled

- 각 상태별 특징:

- submitted: 작업 요청 제출됨
- accepted: 에이전트가 작업 수락
- in\_progress: 작업 진행 중 (진행률 업데이트 가능)
- completed: 작업 완료 (결과물 포함)
- failed: 작업 실패 (에러 정보 포함)
- cancelled: 작업 취소됨

## Message Structure (메시지 구조)

- 에이전트들은 컨텍스트, 응답, 아티팩트, 사용자 지시사항을 전달하기 위해 서로 메시지를 보낼 수 있습니다. 각 메시지는 생성된 이미지와 같은 완전히 형성된 콘텐츠 조각인 "parts"가 포함됩니다

## Message Structure (메시지 구조)

```
{
 "message": {
 "id": "msg_2024_001234",
 "task_id": "task_2024_567890",
 "sender": "marketing_agent",
 "receiver": "analytics_agent",
 "timestamp": "2024-12-31T10:30:00Z",
 "parts": [
 {
 "type": "text",
 "content": "지난 분기 캠페인 성과를 분석해주세요"
 },
 {
 "type": "data",
 "content": {
 "format": "json",
 "data": {...}
 }
 }
]
 }
}
```

## 4. 소통의 표준, 에이전트 프로토콜

### 프로토콜의 필요성

- "똑똑한 AI 비서가 여러 명 있다면, 그들끼리 어떻게 대화하고 협력할 수 있을까?" 라는 질문에서 시작합니다.
- 문제점: 제조사나 개발사마다 AI 에이전트의 '언어'와 '행동 방식'이 다르면 서로 협력할 수 없습니다. 마치 다른 언어를 쓰는 사람들처럼 말이죠.
- 해결책: 에이전트들이 서로의 존재를 인지하고, 능력을 파악하며, 작업을 요청하고 결과를 받을 수 있는 **\*\*표준화된 소통 규약(Protocol)\*\***이 필요합니다.
- 인터넷과의 비유:
  - HTTP: 웹 브라우저와 서버가 정보를 주고받는 표준 규칙
  - SMTP: 이메일 서버들이 메일을 주고받는 표준 규칙
  - A2A 프로토콜: AI 에이전트들이 서로 소통하고 협력하는 표준 규칙

## 4. 소통의 표준, 에이전트 프로토콜

### | Google의 A2A (Agent-to-Agent) 프로토콜

- **개념**: Google이 제안하는 개방형 AI 에이전트 간 통신 프로토콜입니다.
- **비전**: 특정 플랫폼이나 회사에 종속되지 않고, 전 세계의 AI 에이전트들이 서로 원활하게 협력하는 '에이전트 생태계'를 구축하는 것을 목표로 합니다.
- **주요 특징**:
  - 상호운용성(Interoperability): 서로 다른 개발사가 만든 에이전트도 A2A 프로토콜을 따르면 원활하게 통신할 수 있습니다.
  - 작업 위임(Task Delegation): 자신의 능력을 벗어나는 작업은 해당 작업을 가장 잘 처리할 수 있는 다른 에이전트에게 위임할 수 있습니다.
  - 에이전트 탐색(Discovery): 특정 작업을 수행할 수 있는 다른 에이전트를 찾고 발견하는 메커니즘을 제공합니다.

## 4. 소통의 표준, 에이전트 프로토콜

### A2A 프로토콜의 핵심 구성 요소

- 에이전트 식별(Identity): 각 에이전트는 고유한 주소나 ID를 통해 자신을 식별합니다.
- 기능 명세(Capability Schema): 각 에이전트는 자신이 수행할 수 있는 작업(예: 항공권 예약, 날씨 정보 제공)의 종류와 필요한 정보(파라미터)를 표준화된 형식으로 정의하고 공개합니다.
- 의사소통 언어(Communication Language): 작업 요청, 정보 전달, 결과 보고 등을 위한 메시지 형식을 표준화합니다. (예: JSON, gRPC 기반)
- 실행 흐름(Execution Flow): 사용자가 에이전트 A에게 작업을 요청했을 때, 에이전트 A가 에이전트 B에게 하위 작업을 위임하고 결과를 받아 다시 사용자에게 전달하는 일련의 과정을 정의합니다.



# 4. 소통의 표준, 에이전트 프로토콜

## A2A 작동 시나리오 예시: "제주도 여행 계획"

- 사용자: (나의 개인 비서 에이전트에게) "다음 주 금요일에 출발하는 2박 3일 제주도 여행 계획 좀 짜줘."
- 개인 비서 에이전트 (A):
  - 사용자의 요청을 분석합니다. (항공, 숙박, 렌터카 예약 필요)
  - A2A 네트워크에서 각 작업을 가장 잘 수행할 전문 에이전트를 탐색합니다.
- 작업 위임:
  - A -> 항공권 예약 에이전트 (B): "다음 주 금요일 출발, 일요일 도착 제주행 1인 항공권 최저가 검색 및 예약 요청"
  - A -> 숙소 추천 에이전트 (C): "제주공항 근처, 2박, 1인, 평점 4.5 이상 호텔 검색 및 추천 요청"
  - A -> 렌터카 예약 에이전트 (D): "제주공항에서 3일간 소형차 렌트 최저가 검색 및 예약 요청"
- 결과 취합:
  - B, C, D 에이전트는 각각 작업을 수행하고 결과를 표준화된 형식으로 A에게 회신합니다.
- 최종 보고:
  - A 에이전트는 취합된 정보를 바탕으로 최종 여행 계획(항공편, 숙소, 렌터카 정보 및 총비용)을 정리하여 사용자에게 제시합니다.

## 5. A2A가 바꿀 미래와 우리의 준비

### | 비즈니스 및 산업에 미치는 영향

- 비스의 결합: 단일 앱이나 서비스가 아닌, 여러 에이전트가 협력하여 사용자에게 끊김 없는(Seamless) 통합 경험을 제공합니다.
- 새로운 비즈니스 모델: 특정 기능에 고도로 특화된 '전문 에이전트'를 개발하고 판매하는 시장이 형성될 수 있습니다.  
(예: 법률 자문 에이전트, 의료 진단 보조 에이전트)
- 하이퍼-오토메이션(Hyper-automation): 기업의 복잡한 워크플로우(예: 공급망 관리, 고객 지원)가 다수의 전문 에이전트 협력을 통해 완전히 자동화될 수 있습니다.

## 5. A2A가 바꿀 미래와 우리의 준비

### | 우리가 준비해야 할 것

- 기술적 관점: 표준 프로토콜에 대한 이해와 이를 기반으로 에이전트를 개발할 수 있는 역량이 필요합니다.
- 기획적 관점: 단일 기능이 아닌, 다른 에이전트와 어떻게 '협력'하여 더 큰 가치를 만들 수 있을지 고민하는 생태계적 사고가 중요합니다.
- 보안 및 신뢰: 에이전트 간의 데이터 교환 시 발생할 수 있는 보안 문제, 개인정보 보호, 그리고 위임된 작업의 신뢰성을 확보하는 방안에 대한 고민이 필요합니다.

# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약





## 학습목표

- 이 워크샵에서는 011ama 에 확인할 수 있다

## 눈높이 체크

- 011ama 를 알고 계시나요?



# 1. Ollama?

## Ollama?

- Ollama는 Open Source Large Language Model Meta AI의 약자로, 말 그대로 개방형 대규모 언어 모델을 의미함. 쉽게 말해, 누구나 자신의 컴퓨터에서 대규모 언어 모델을 직접 실행해 볼 수 있도록 만들어진 오픈 소스 플랫폼이라고 할 수 있다.
- Ollama의 특징
  - 오픈 소스: 누구나 소스 코드를 자유롭게 확인하고 수정할 수 있다.
  - 개인용 LLM: 강력한 컴퓨터가 없어도 개인 PC에서 LLM을 경험할 수 있다.
  - 사용 편의성: 복잡한 설정 없이 간편하게 LLM을 설치하고 사용할 수 있다.
  - 확장성: 다양한 플랫폼과 라이브러리와의 통합을 지원하여 활용도가 높다.



# 1. Ollama?

## Ollama?

- Ollama를 사용하면 무엇을 할 수 있을까요?
  - 텍스트 생성: 다양한 주제의 텍스트를 생성하거나 번역할 수 있다.
  - 대화: 챗봇과 같은 대화형 시스템을 구축할 수 있다.
  - 코드 생성: 코드를 자동으로 생성하거나 설명할 수 있다.
  - 정보 검색: 질문에 대한 답변을 찾거나 지식을 탐색할 수 있다.
- Ollama의 장점
  - 낮은 진입 장벽: 복잡한 설정 없이 누구나 쉽게 시작할 수 있다.
  - 높은 커스터마이징: 모델을 자유롭게 수정하여 자신만의 LLM을 만들 수 있다.
  - 지속적인 발전: 오픈 소스 커뮤니티를 통해 지속적으로 발전하고 있다.
- Ollama의 단점
  - 성능: 상용 클라우드 기반 LLM에 비해 성능이 다소 떨어질 수 있다.
  - 학습 데이터: 학습 데이터의 양과 질에 따라 모델의 성능이 크게 좌우됩니다.



# 1. Ollama?

## Ollama?

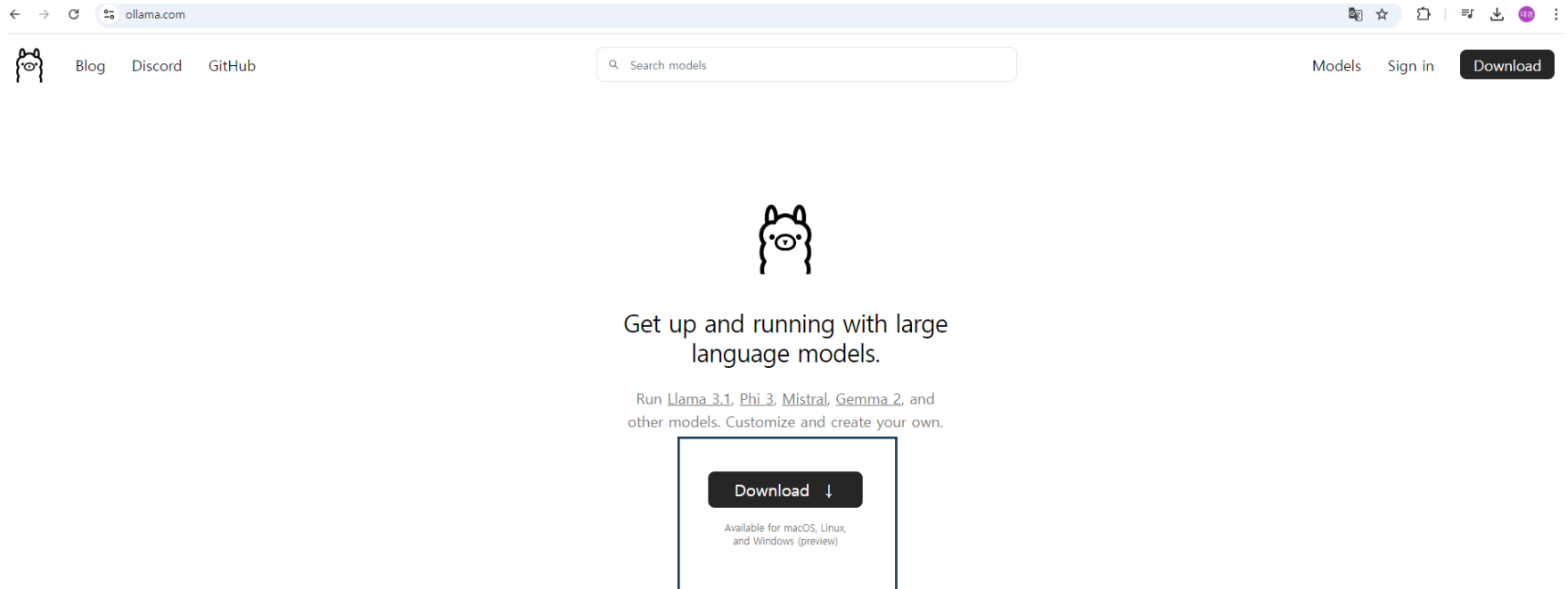
- Ollama 활용 방법
- Ollama를 사용하려면 다음과 같은 방법을 활용할 수 있다.
- 직접 설치: Ollama 공식 문서를 참고하여 직접 설치하고 사용할 수 있다.
- Docker 이미지 사용: Docker 이미지를 이용하여 간편하게 설치하고 실행할 수 있다.
- API 활용: Ollama는 REST API를 제공하여 다른 프로그램에서 모델을 호출하고 사용할 수 있다.



## 2. Ollama 사용

### Ollama 설치

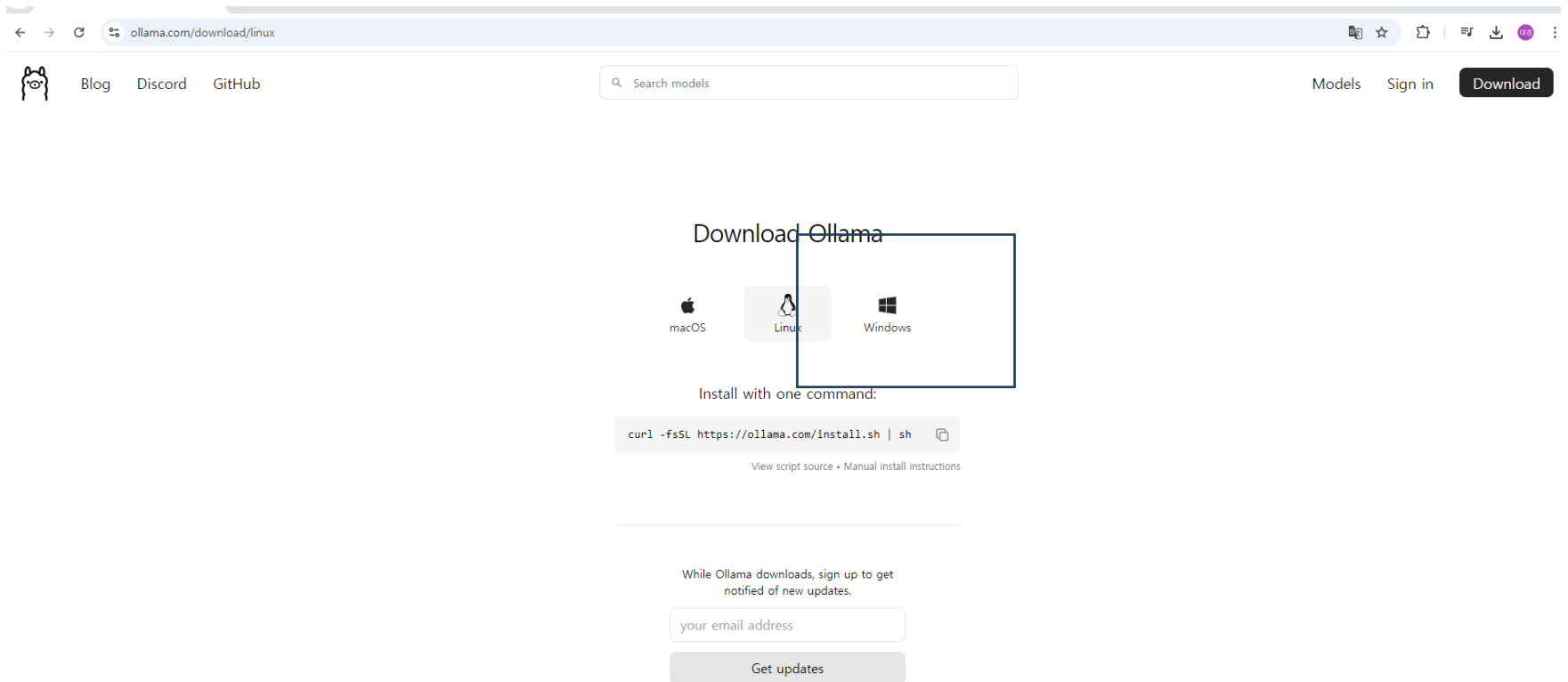
- <https://ollama.com/>



# 2. Ollama 사용

## Ollama 설치

- <https://ollama.com/>



The screenshot shows the Ollama website's download page for Linux. The browser's address bar displays "ollama.com/download/linux". The page features a navigation bar with links to "Blog", "Discord", and "GitHub", a search bar labeled "Search models", and buttons for "Models", "Sign in", and "Download". The main content area is titled "Download Ollama" and includes icons for "macOS", "Linux", and "Windows". A box highlights the "Linux" icon. Below this, the text "Install with one command:" is followed by a code block containing the command: `curl -fsSL https://ollama.com/install.sh | sh`. Links for "View script source" and "Manual install instructions" are provided. At the bottom, a section prompts users to sign up for updates, featuring a text input field labeled "your email address" and a "Get updates" button.

ollama.com/download/linux

Blog Discord GitHub

Search models

Models Sign in Download

### Download Ollama

macOS Linux Windows

Install with one command:

```
curl -fsSL https://ollama.com/install.sh | sh
```

[View script source](#) • [Manual install instructions](#)

While Ollama downloads, sign up to get notified of new updates.

your email address

Get updates



## 2. Ollama 사용

### Ollama 설치

- <https://ollama.com/>

PC > 다운로드 >

이름

▼ 오늘 (1)

 OllamaSetup.exe

... 더보기 >

### Windows의 PC 보호

Microsoft Defender SmartScreen에서 인식할 수 없는 앱의 시작을 차단했습니다. 이 앱을 실행하면 PC가 위험에 노출될 수 있습니다.

앱: OllamaSetup.exe

게시자: Ollama Inc.

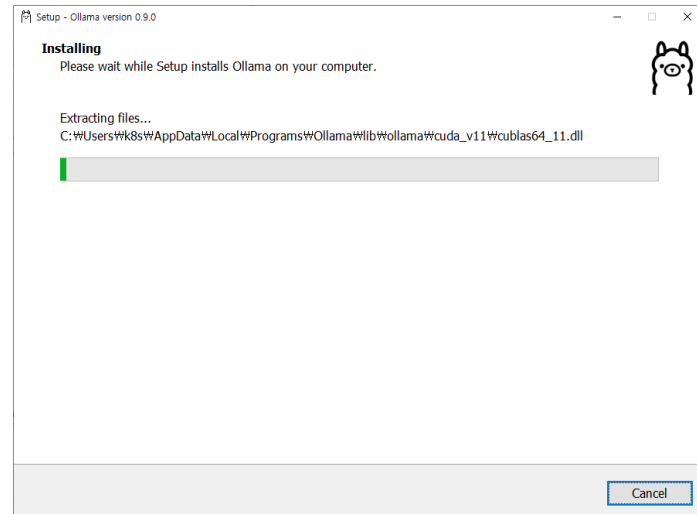
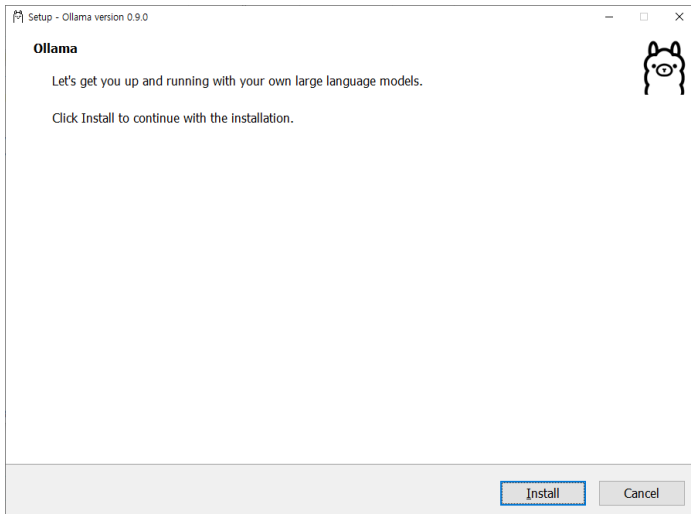
실행

실행 안 함

## 2. Ollama 사용

### Ollama 설치

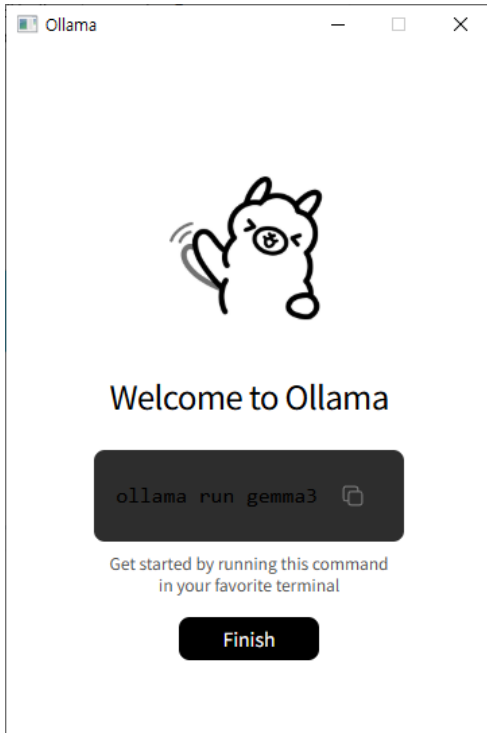
- <https://ollama.com/>



## 2. Ollama 사용

### Ollama 설치

- <https://ollama.com/>



## 2. Ollama 사용

### Ollama 명령어

```
c:\DEV>ollama
```

Usage:

```
ollama [flags]
ollama [command]
```

Available Commands:

serve	Start ollama
create	Create a model from a Modelfile
show	Show information for a model
run	Run a model
stop	Stop a running model
pull	Pull a model from a registry
push	Push a model to a registry
list	List models
ps	List running models
cp	Copy a model
rm	Remove a model
help	Help about any command

Flags:

-h, --help	help for ollama
-v, --version	Show version information

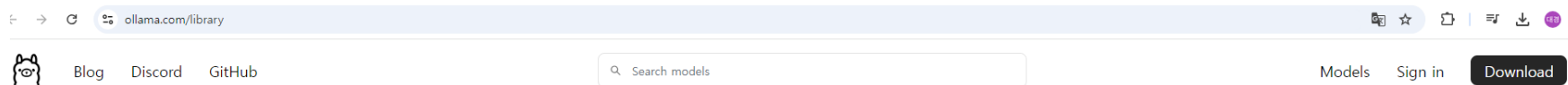
Use "ollama [command] --help" for more information about a command.

```
c:\DEV>
```

## 2. Ollama 사용

### Ollama 다운로드 받은 모델 조회

<https://ollama.com/library>



Models

Filter by name...

Featured

#### Llama3.1

Llama 3.1 is a new state-of-the-art model from Meta available in 8B, 70B and 405B parameter sizes.

Tools 8B 70B 405B

↓ 5.4M Pulls    🔖 94 Tags    ⌚ Updated 7 days ago

#### gemma2

Google Gemma 2 is a high-performing and efficient model

## 2. Ollama 사용

### Ollama 다운로드 받은 모델 조회

<https://ollama.com/library>

Model	Parameters	Size	Download Command
Llama 3	8B	4.7GB	<code>ollama run llama3</code>
Llama 3	70B	40GB	<code>ollama run llama3:70b</code>
Phi-3	3.8B	2.3GB	<code>ollama run phi3</code>
Mistral	7B	4.1GB	<code>ollama run mistral</code>
Neural Chat	7B	4.1GB	<code>ollama run neural-chat</code>
Starling	7B	4.1GB	<code>ollama run starling-lm</code>
Code Llama	7B	3.8GB	<code>ollama run codellama</code>
Llama 2 Uncensored	7B	3.8GB	<code>ollama run llama2-uncensored</code>
LLaVA	7B	4.5GB	<code>ollama run llava</code>
Gemma	2B	1.4GB	<code>ollama run gemma:2b</code>
Gemma	7B	4.8GB	<code>ollama run gemma:7b</code>
Solar	10.7B	6.1GB	<code>ollama run solar</code>



## 2. Ollama 사용

### Ollama 다운로드 받은 모델 조회

추천하고 싶은 한글이 어느정도 되는 모델들은 아래와 같다

```
$ ollama run gemma:instruct (한글이 그래도 나온다!)
```

```
$ ollama run llama3:instruct (한글로 설명해달라고 하면 제법 한다)
```

```
$ ollama run command-r (cohere의 모델인데 20b이지만 한글이 꽤 된다)
```

```
$ ollama run codegemma:instruct
```



Year	Percentage
2020	25%
2021	65%

```
pulling manifest
pulling 6a0746a1ec1a: 30% ██████████ | 1.4 GB/4.7 GB
55 MB/s 59s
pulling manifest
pulling 6a0746a1ec1a: 100% ██ 4.7 GB
pulling 4fa551d4f938: 100% ██ 12 KB
pulling 8ab4849b038c: 100% ██ 254 B
pulling 577073ffcc6c: 100% ██ 110 B
pulling 3f8eb4da87fa: 100% ██ 485 B
pulling manifest
pulling 6a0746a1ec1a: 100% ██ 4.7 GB
pulling 4fa551d4f938: 100% ██ 12 KB
pulling 8ab4849b038c: 100% ██ 254 B
pulling 577073ffcc6c: 100% ██ 110 B
pulling 3f8eb4da87fa: 100% ██ 485 B
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

## 2. Ollama 사용

### llama3

```
>>> 서울의 관광지를 소개해 줘
```

```
writing manifest
```

```
success
```

```
Seoul! The capital city of South Korea is a treasure trove of
```

Ctrl + C

```
>>> /bye
```

```
c:\DEV>
```



## 2. ollama 사용

### llama3

```
c:\DEV>ollama list
```

NAME	ID	SIZE	MODIFIED
llama3:latest	365c0bd3c000	4.7 GB	3 minutes ago

```
c:\DEV>
```

## 2. ollama 사용

### Gemma

Gemma is a family of lightweight, state-of-the-art open models built by Google DeepMind. Updated to version 1.1

ollama.com/library

#### gemma

Gemma is a family of lightweight, state-of-the-art open models built by Google DeepMind. Updated to version 1.1

2B 7B

↓ 4.1M Pulls   ↗ 102 Tags   ⌚ Updated 5 months ago

#### qwen

Qwen 1.5 is a series of large language models by Alibaba Cloud spanning from 0.5B to 110B parameters

0.5B 1.8B 4B 32B 72B 110B

↓ 4M Pulls   ↗ 379 Tags   ⌚ Updated 3 months ago

# 2. ollama 사용

## Gemma

```
ollama run gemma:latest
```

ollama.com/library/gemma:latest

Models Sign in Download

Search models

### gemma

Gemma is a family of lightweight, state-of-the-art open models built by Google DeepMind. Updated to version 1.1

2B 7B

4.1M Pulls Updated 5 months ago

latest 102 Tags ollama run gemma:latest

Updated 5 months ago		a72c7f4d0a15 · 5.0GB
model	arch <b>gemma</b> · parameters <b>8.54B</b> · quantization <b>Q4_0</b>	5.0GB
license	Gemma Terms of Use Last modified: February 21, 2024 By using, re...	8.4kB
template	<start_of_turn>user {{ if .System }}{{ .System }} {{ end }}{{ .P...	136B
params	{ "penalize_newline": false, "repeat_penalty": 1, "stop": [ "<start_of_...	109B

Readme

## 2. ollama 사용

### Gemma

```
ollama run gemma:latest
```

```
c:\DEV>ollama run gemma:latest
```

```
pulling manifest
```

```
pulling ef311de6af9d: 100%
```

```
5.0 GB
```

```
pulling 097a36493f71: 100%
```

```
8.4 KB
```

```
pulling 109037bec39c: 100%
```

```
136 B
```

```
pulling 65bb16cf5983: 100%
```

```
109 B
```

```
pulling 0c2a5137eb3c: 100%
```

```
483 B
```

```
verifying sha256 digest
```

```
writing manifest
```

```
success
```

```
>>> /bye
```

```
c:\DEV>
```

# 3. Ollama 가상환경

## Ollama 설치

```
c:\DEV\envs>python -m venv py3_10_ollama
c:\DEV\envs>cd py3_10_ollama
c:\DEV\envs\py3_10_ollama>Scripts\activate.bat
(py3_10_ollama) c:\DEV\envs\py3_10_ollama>pip install ollama langchain beautifulsoup4
chromadb gradio
Collecting ollama
 Downloading ollama-0.5.1-py3-none-any.whl (13 kB)
Collecting langchain
 Using cached langchain-0.3.25-py3-none-any.whl (1.0 MB)
Collecting beautifulsoup4

...

(py3_10_ollama) c:\DEV\envs\py3_10_ollama>pip install langchain_community

...

(py3_10_ollama) c:\DEV\envs\py3_10_ollama>pip install langchain_ollama

...

(py3_10_ollama) c:\DEV\envs\py3_10_ollama>python.exe -m pip install --upgrade pip
```



# 3. Ollama 가상환경

## 쥬피터 노트북 내보내기

```
(py3_10_ollama) c:\DEV\envs\py3_10_ollama>pip install ipykernel
```

```
Collecting ipykernel
```

```
Using cached ipykernel-6.29.5-py3-none-any.whl.metadata (6.3 kB)
```

```
...
```

```
(py3_10_ollama) c:\DEV\envs\py3_10_ollama>python -m ipykernel install --user --name
py3_10_ollama --display-name "py3_10_ollama"
```

```
Installed kernelspec py3_10_ollama in C:\Users\k8s\AppData\Roaming\jupyter\kernels\py3_10_ollama
```

```
(py3_10_ollama) c:\DEV\envs\py3_10_ollama> pip install -qU pymupdf
```

# 3. Ollama 가상환경

## 샘플 1-gradio

```
(py3_10_ollama) c:\DEV\envs\py3_10_ollama>cd ..
```

```
(py3_10_ollama) c:\DEV\envs>cd ..
```

```
(py3_10_ollama) c:\DEV>cd SamsungDisplay
```

```
(py3_10_ollama) c:\DEV\SamsungDisplay >dir/w/p
```

C 드라이브의 볼륨에는 이름이 없습니다.

볼륨 일련 번호: BED0-C858

c:\DEV\PycharmProjects 디렉터리

[.]	[..]	[.idea]	[Week_1]	[ai_analysis]	
[data_pre_processing]	[flask_works]	[ml_analysis]	[python_works]	[statistical_analysis]	
0개 파일	0 바이트				
10개 디렉터리	10,669,080,576 바이트	남음			

```
(py3_10_ollama) c:\DEV\SamsungDisplay >cd Week_1
```

```
(py3_10_ollama) c:\DEV\SamsungDisplay\Week_1\Ollama_pro>ollama pull mxbai-embed-large
```

ollama pull mxbai-embed-large



# 3. Ollama 가상환경

## 샘플 1-gradio

```
Ollama_Test.py
import gradio as gr
import bs4
import ollama
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import Chroma
from langchain_ollama import OllamaEmbeddings

문서 로드 및 벡터화
def load_and_retrieve_docs(url: str):
 loader = WebBaseLoader(
 web_paths=(url,),
 bs_kwargs=dict() # BeautifulSoup 기본 설정
)
 docs = loader.load()

 text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
 splits = text_splitter.split_documents(docs)

 embeddings = OllamaEmbeddings(model="mxbai-embed-large")
 vectorstore = Chroma.from_documents(documents=splits, embedding=embeddings)

 return vectorstore.as_retriever()

문서 포매팅
def format_docs(docs):
 return "\n\n".join(doc.page_content for doc in docs)
```

# 3. Ollama 가상환경

## 샘플1-gradio

# RAG 체인 동작

```
def rag_chain(url: str, question: str) -> str:
 retriever = load_and_retrieve_docs(url)
 retrieved_docs = retriever.invoke(question)

 context = format_docs(retrieved_docs)
 prompt = f"Question: {question}\n\nContext: {context}"

 response = ollama.chat(
 model='llama3',
 messages=[
 {
 "role": "system",
 "content": "You are a helpful assistant. Check the url content and answer the question. Translate the answer in Korean with emoji."
 },
 {
 "role": "user",
 "content": prompt
 }
]
)

 return response['message']['content']
```

# 3. Ollama 가상환경

## 샘플 1-gradio

```
Gradio 인터페이스
iface = gr.Interface(
 fn=rag_chain,
 inputs=["text", "text"],
 outputs="text",
 title="🦙 LLaMA 3 - RAG 기반 질문 응답",
 description="웹페이지 URL과 질문을 입력하면, 해당 내용을 기반으로 LLaMA 3가 한국어 답변을 제공합니다. KR ✨"
)

if __name__ == "__main__":
 iface.launch()
```

# 3. Ollama 가상환경

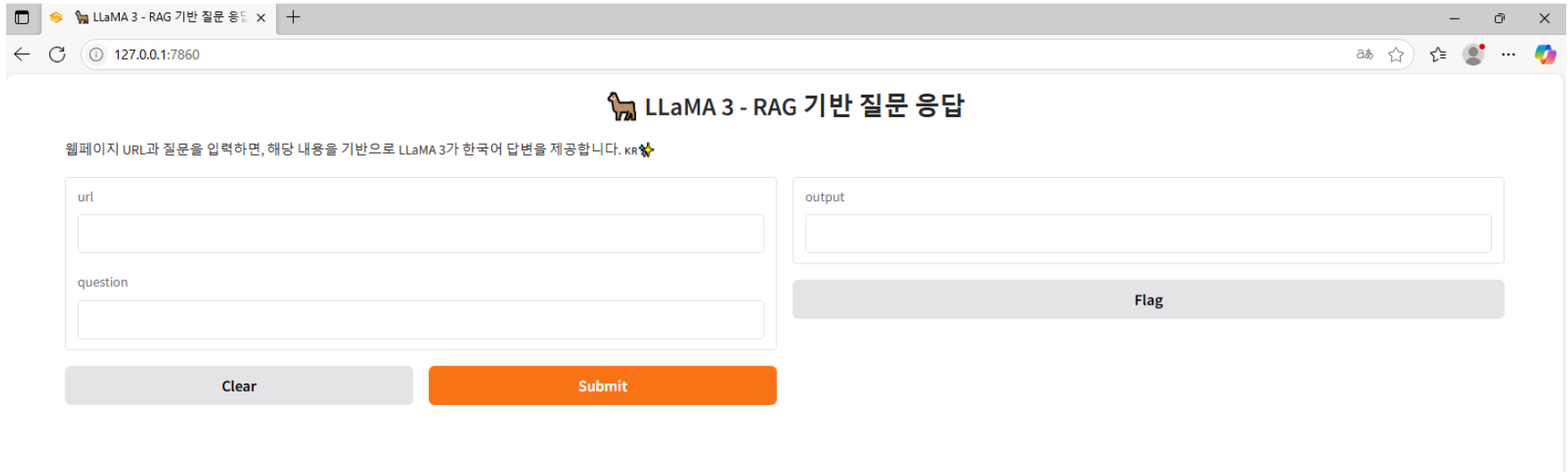
## 샘플1-gradio

(py3\_10\_ollama) c:\DEV\SamsungDisplay\Week\_1\Ollama\_pro>python Ollama\_Test.py  
USER\_AGENT environment variable not set, consider setting it to identify your requests.

\* Running on local URL: <http://127.0.0.1:7860>

- To create a public link, set `share=True` in `launch()`.

- <http://127.0.0.1:7860/>



The screenshot shows a web browser window with the address bar set to [127.0.0.1:7860](http://127.0.0.1:7860). The page title is "LLaMA 3 - RAG 기반 질문 응답". Below the title, there is a subtitle in Korean: "웹페이지 URL과 질문을 입력하면, 해당 내용을 기반으로 LLaMA3가 한국어 답변을 제공합니다. KR 🌟". The interface contains two input fields on the left: "url" and "question". To the right of these is a larger "output" field. At the bottom left, there are two buttons: "Clear" (grey) and "Submit" (orange). At the bottom right, there is a grey button labeled "Flag".

# 3. Ollama 가상환경

## 샘플 1-gradio

<https://python.langchain.com/v0.2/docs/introduction/>

A newer LangChain version is out! Check out the [latest version](#).

**LangChain** Integrations API reference More

v0.2 Search

**Introduction**

Tutorials

How-to guides

Conceptual guide

Ecosystem

LangSmith

LangGraph

Versions

Overview of v0.2

Release policy

Pydantic compatibility

Migrating to v0.2

Migrating from v0.0 chains

Security

This is documentation for LangChain v0.2, which is no longer actively maintained.

For the current stable version, see [this version](#) (Latest).

Introduction

**Introduction**

LangChain is a framework for developing applications powered by large language models (LLMs).

LangChain simplifies every stage of the LLM application lifecycle:

- Development:** Build your applications using LangChain's open-source [building blocks](#), [components](#), and [third-party integrations](#). Use [LangGraph](#) to build stateful agents with first-class streaming and human-in-the-loop support.
- Productionization:** Use [LangSmith](#) to inspect, monitor and evaluate your chains, so that you can continuously optimize and deploy with confidence.
- Deployment:** Turn your LangGraph applications into production-ready APIs and Assistants with [LangGraph Cloud](#).

Deployment

LangGraph Cloud

LangSmith

Tutorials

How-to guides

Conceptual guide

API reference

Ecosystem

LangSmith

LangGraph

Additional resources

Versions

Security

Integrations

Contributing

# 3. Ollama 가상환경

## 샘플1-gradio

<https://python.langchain.com/v0.2/docs/introduction/>

LangChain 프레임워크에 대해 설명해줘.

◦ 내용을 입력한다.

127.0.0.1:7860

Enter a URL and a query to get answers from the RAG chain.

url

<https://python.langchain.com/v0.2/docs/introduction/>

question

[LangChain 프레임워크에 대해 설명해 줘.](#)

Clear

Submit

output

Here's the answer:

**\*\*LangChain 프레임워크는-large language model(LLM)을 기반으로 하는 애플리케이션을 개발하는 데 사용되는 프레임워크입니다. LangChain은 LLM 애플리케이션의 수명 주기를 간소화합니다.\*\***

Translated in Korean with emojis:

 LangChain 프레임워크는-large language model(LLM)을 기반으로 하는 애플리케이션을 개발하는 데 사용되는 프레임워크입니다.  LangChain은 LLM 애플리케이션의 수명 주기를 간소화합니다. 



## 4. 실습

### 실습 24 : PDF 파일을 첨부하여 처리하는 코드

#### ● 변경 핵심

- WebBaseLoader → PyMuPDFLoader로 교체하여 PDF 문서 로딩
- Gradio에서 파일 업로드 지원
- 기존 URL 입력 제거, 대신 PDF 업로드로 RAG 실행
- 나머지 Ollama, LLaMA3, Chroma 벡터스토어 등은 유지

#### ● 사용 방법 요약

- 실행 후 Gradio UI에서 PDF 업로드 + 질문 입력
- 내부적으로 PyMuPDFLoader로 PDF → 문서화 → 임베딩 벡터화 → RAG
- 결과는 LLaMA3가 한국어로 출력

#### ● 필요 패키지 설치 (미설치 시)

`pip install gradio langchain langchain-community langchain-ollama pymupdf chromadb`

#### ● Ollama 실행

`ollama pull mxbai-embed-large`

# 『3과목』

## RAG & 고급 프롬프트

- Part 1. 오리엔테이션
- Part 2. Langchain 전 프롬프트 엔지니어링
- Part 3. Langchain 기본
- Part 4. AI 에이전트 개발
- Part 5. AI 에이전트 프로토콜 : MCP와 A2A
- Part 6. Ollama
- Part 7. 마무리 및 요약





# 1. 핵심 요약

## 학습 내용

- 랭체인과 RAG에 대해 학습

핵심 주제	핵심 개념
RAG	검색+생성 결합
Hallucination	사실 검증 구조
템플릿 설계	업무 자동화 기반
실무 적용	RAG+Prompt 통합



## 2. 다음 과목 예고

### Project

- Cursor.AI + LangChain + GitHub을 활용해 AI 자동화 워크플로우를 직접 구축하고 버전 관리까지 실습하는 캡스톤 과정

# THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

- 🎯 목표 달성을 위한 여정이 시작됩니다.
- 🌟 궁금한 점이 있으시면 언제든지 문의해주세요!
- 🚀 함께 코더와 프롬프트 전문가로 성장해 나갑시다!